Reward variation without demonstrated token savings: audited fixed-reward studies

Arya Somu

October 11, 2026

Two fixed-reward studies are complete and independently audited: a historical one-seed development contrast and a separately frozen three-seed mechanism intervention. Neither meets its registered practical token-response criterion. This paper preserves the derivations, negative findings and recovery history. Adaptive reward superiority and generalization remain unestablished. The scientific source and records for both completed studies accompany the versioned research release.

Abstract

We study how a fixed token-cost penalty changes learning support and final behavior in a small coding model. A mathematical derivation separates an ideal adaptive preference controller from the implemented leave-one-out policy-gradient estimator, LoRA parameterization and Adam optimizer. An earlier audited one-seed development comparison produced more penalty updates but no token savings. We then froze a separate three-arm mechanism study with fresh controls, three paired training seeds and 96 previously untouched synthetic repair tasks. Nine single-A40 workers trained Qwen3-4B-Instruct-2507 adapters for 64 groups of four fresh trajectories each. Arms used success-only reward (A), success minus tokens/1,024 (B), or the same penalty with a complete optimizer skip on homogeneous-success groups (C). Each final policy generated two samples per task. The primary B-C contrast was -29.56 tokens (-1.80% relative to C; descriptive 95% crossed seed/task interval -76.66 to +15.48) and +3.99 accuracy percentage points (-0.52 to +8.51). B used fewer mean tokens in each seed, but the interval crossed zero and the registered 10% promising-response criterion was not met. B-A was -16.15 tokens (-58.62 to +25.32). Across three seeds, A, B and C took 96, 174 and 95 optimizer updates, respectively; all 365 active updates were clipped. C skipped 97 homogeneous-success groups. Its intervention changes update support, Adam moments and future trajectories, so the endpoint contrast does not isolate a single unbiased cost-gradient contribution. All nine complete terminal archives passed streamed hash/size verification and independent frozen-source CPU audits. Three seeds, a restricted synthetic task language, frequent generation ceilings and final-only evaluation limit interpretation. The study supplies a reproducible mechanism intervention and a negative practical decision, without establishing absence of a small effect, adaptive superiority or repository-level coding generalization.

1 Introduction

Model capability is often reported independently of inference cost. For a deployed policy, both solve probability and generated computation matter. A linear utility, $R = S - \lambda C$, makes that preference explicit, but a fixed penalty does not directly specify the direction of improvement relative to an initial policy.

Lakkapragada proposes adapting the penalty to balance relative success gains and cost reductions. His controlled environment has five trainable parameters and a known success-cost relation-

ship. Our question is how this idea behaves when the policy is a language model trained through a small LoRA adapter, with variable-length sampled reasoning and stochastic optimization.

We separate three questions: whether the numerical estimator is faithful to the sampled policy; whether a fixed cost preference produces an observable behavioral response; and whether adaptive preferences steer that response on untouched tasks across training seeds. Each requires different evidence. Infrastructure correctness and improved prompting alone do not answer the final question.

Our contribution is a transparent mathematical account, an audited one-seed exploratory contrast and a separately frozen three-seed optimizer intervention, including failed diagnostics and interruptions. We do not claim a new reward algorithm or a superior coding policy. REINFORCE-style language-model optimization has prior empirical support in other alignment settings [1]. Work on length penalties also identifies optimization and sparse-signal issues [2]; its tasks and training recipe differ from ours. Those results motivate examining the estimator and optimizer, not transferring their reported gains to this experiment.

2 Objective and adaptive update

This section supplies the complete interior derivation of the controller adapted from Lakkapragada. It is an ideal policy-space construction. The empirical studies test fixed utilities and a masked optimizer intervention; an adaptive-controller experiment remains prospective.

2.1 Reader's guide and mathematical prerequisites

The intended baseline is elementary probability, derivatives, and matrix multiplication. We use natural logarithms throughout. $\mathbb{E}[U]$ denotes an average over the specified random experiment; $\mathrm{Var}(U) = \mathbb{E}[(U-\mathbb{E}U)^2]$ and $\mathrm{Cov}(U,V) = \mathbb{E}[(U-\mathbb{E}U)(V-\mathbb{E}V)]$. A gradient $\nabla_\theta f$ collects the partial derivatives of a scalar f with respect to the trainable parameters θ. A conditional expectation holds the variables following the conditioning bar fixed. The symbol $\widehat{\ }$ marks an estimate computed from observations. An arg min returns a minimizing argument, whereas min returns the minimum value.

The derivation follows this chain: specify a common success–cost improvement; rewrite its two constraints as a minimum; express that minimum using a mixture weight; identify the corresponding linear reward; solve one regularized mixture update; estimate the reward gradient from discrete samples; apply it through LoRA and the optimizer; and compare complete final-policy evaluations. Equations describe these transitions explicitly. They do not supply a closed-form solution for the trained language model.

Symbol	Meaning
x, e, y	Task, effort condition, and generated trajectory
π, θ	Policy distribution and its trainable adapter parameters
S, C, s, c	Realized success/cost and their policy expectations
s_0, c_0, u, v	Fixed baseline expectations and relative improvements
$\tau, \alpha, \beta, \lambda, \eta$	Common improvement, preference, dual mixture, token penalty, controller step size
G, i, A_i, h_i	Training-group size, sample index, advantage, trajectory score gradient
k, t	Completed training-group count and actual optimizer-update count
N, M, j, m	Evaluation tasks, samples per task, task index, sample index
B	Bootstrap replicate count; in Appendix A, a separately local computation-block count

The LoRA matrices also use A, B; those matrix symbols are distinct from sampled answer A, scalar advantage A_i, and bootstrap count B. Their types and subscripts identify their roles. Subscript e is omitted when one effort condition is held fixed. Later proofs invoke the ordinary product/chain

rules, finite-sum differentiation, and independence; the transformer architecture and these elementary calculus rules are prerequisites rather than new results proved here.

2.2 Probability space, baseline and units

Let $X \sim D_e$ be a task at effort condition e, and let Y be a generated trajectory under policy π. The hidden-example evaluator produces $S(X,Y) \in \{0,1\}$; generated-token cost is $C(Y) \geq 0$. Define

$$s_e(\pi) = \mathbb{E}_{X \sim D_e,\, Y \sim \pi(\cdot|X,e)}[S(X,Y)], \tag{1}$$

$$c_e(\pi) = \mathbb{E}_{X \sim D_e,\, Y \sim \pi(\cdot|X,e)}[C(Y)]. \tag{2}$$

The original policy π_0 fixes $s_{e,0} = s_e(\pi_0) > 0$ and $c_{e,0} = c_e(\pi_0) > 0$. Task distribution, scoring, token counting and sampling settings are held fixed. Relative gains are

$$u_e(\pi) = \frac{s_e(\pi) - s_{e,0}}{s_{e,0}}, \qquad v_e(\pi) = \frac{c_{e,0} - c_e(\pi)}{c_{e,0}}. \tag{3}$$

Thus $u_e > 0$ means improved solve probability and $v_e > 0$ means reduced token cost. Both are dimensionless. A penalty λ_e has units of success probability per generated token. For a penalty held fixed during a policy update, linearity gives

$$R_{\lambda_e}(X,Y) = S(X,Y) - \lambda_e C(Y), \qquad J_{\lambda_e}(\pi) = \mathbb{E}[R_{\lambda_e}] = s_e(\pi) - \lambda_e c_e(\pi). \tag{4}$$

The baseline anchor is the original policy, not the preceding training checkpoint.

2.3 Constrained common improvement

For an interior preference $0 < \alpha < 1$, consider

$$\begin{aligned} \max_{\pi,\tau} \quad & \tau \\ \text{subject to} \quad & u_e(\pi) \geq \alpha\tau, \\ & v_e(\pi) \geq (1-\alpha)\tau. \end{aligned} \tag{5}$$

For any fixed policy, the largest feasible common improvement is

$$\tau^*(\pi) = \min\left\{ \frac{u_e(\pi)}{\alpha}, \frac{v_e(\pi)}{1-\alpha} \right\}. \tag{6}$$

Indeed, division by the positive numbers α and $1-\alpha$ turns the constraints into $\tau \leq u_e/\alpha$ and $\tau \leq v_e/(1-\alpha)$. Both must hold, so the smaller upper bound is binding. This reasoning also applies when either gain is negative. Multiplying by the positive, policy-independent constant $\alpha(1-\alpha)$ leaves the maximizing policy unchanged:

$$\max_{\pi} \tau^*(\pi) \quad \Longleftrightarrow \quad \max_{\pi} \min\{(1-\alpha)u_e(\pi),\, \alpha v_e(\pi)\}. \tag{7}$$

The original policy has $u_e = v_e = 0$, so it is a feasible reference with $\tau = 0$. Increasing α increases the success component required per unit of common improvement.

2.4 Dual mixture and scalarization

For real numbers a, b, a linear function over a closed interval attains its minimum at an endpoint, hence

$$\min\{a, b\} = \min_{0\le\beta\le 1} \{\beta a + (1-\beta)b\}. \tag{8}$$

To see the endpoints directly, write $\beta a + (1-\beta)b = b + \beta(a-b)$. If $a \ge b$, its slope is nonnegative and $\beta = 0$ gives b. If $a \le b$, $\beta = 1$ gives a. If $a = b$, every mixture attains the same value. Applying this identity to Equation (7) yields

$$\max_{\pi} \min_{\beta\in[0,1]} L_e(\pi, \beta), \qquad L_e(\pi, \beta) = \beta(1-\alpha)u_e(\pi) + (1-\beta)\alpha v_e(\pi). \tag{9}$$

$$L_e(\pi, \beta) = \frac{\beta(1-\alpha)}{s_{e,0}} s_e(\pi) - \frac{(1-\beta)\alpha}{c_{e,0}} c_e(\pi) + \alpha - \beta. \tag{10}$$

The term $\alpha - \beta$ is constant in π at fixed β, but its derivative in β is -1. It cannot be discarded when updating the dual mixture.

The expansion uses the following intermediate substitutions:

$$L_e = \beta(1-\alpha)\left(\frac{s_e}{s_{e,0}} - 1\right) + (1-\beta)\alpha\left(1 - \frac{c_e}{c_{e,0}}\right), \tag{11}$$

$$-\beta(1-\alpha) + (1-\beta)\alpha = -\beta + \alpha\beta + \alpha - \alpha\beta = \alpha - \beta. \tag{12}$$

For fixed interior α, β, multiplying Equation (10) by $s_{e,0}/[\beta(1-\alpha)] > 0$ gives a policy objective equivalent to Equation (4), with

$$\boxed{\lambda_e = \frac{s_{e,0}}{c_{e,0}} \frac{\alpha}{1-\alpha} \frac{1-\beta}{\beta}.} \tag{13}$$

In particular, after multiplication the full expression is

$$\frac{s_{e,0}}{\beta(1-\alpha)} L_e = s_e - \underbrace{\frac{s_{e,0}(1-\beta)\alpha}{\beta(1-\alpha)c_{e,0}}}_{\lambda_e} c_e + \frac{s_{e,0}(\alpha-\beta)}{\beta(1-\alpha)}. \tag{14}$$

The last summand does not vary with the policy at a fixed mixture, which is exactly why the remaining policy-dependent utility is $s_e - \lambda_e c_e$. This rescaling is valid only for optimizing π with β fixed. The scale and discarded additive term depend on β, so differentiating the rescaled policy utility in β would not recover the original dual update. No interchange of $\max_\pi$ and $\min_\beta$ is assumed.

2.5 Shared parameters across effort settings

The preceding scalarization is exact for one fixed effort condition. A shared adapter couples policies across prompts. If a hypothetical joint objective weights effort conditions by w_e, $\sum_e w_e = 1$, then for fixed interior mixtures

$$L_{\text{joint}}(\pi, \boldsymbol{\beta}) = \sum_e w_e \left[a_e J_{\lambda_e}(\pi) + \alpha - \beta_e\right], \tag{15}$$

$$a_e = \frac{\beta_e(1-\alpha)}{s_{e,0}}. \tag{16}$$

Replacing this policy objective with $\sum_e w_e J_{\lambda_e}(\pi)$ removes effort-dependent positive scales and need not preserve its maximizing policy. Per-effort scalarization alone therefore does not establish equivalence for an arbitrary mixture of prompts sharing parameters. The current contrast trains reasoning at a single effort condition; it does not solve a joint multi-effort frontier objective.

2.6 KL-regularized dual step, including its minimizer

At controller iteration t, hold observed gains $\widehat{u}_{e,t}, \widehat{v}_{e,t}$ fixed. For $\eta > 0$ and $0 < \beta_t < 1$, define

$$\beta_{t+1} = \arg\min_{\beta \in [0,1]} F_t(\beta), \tag{17}$$

$$F_t(\beta) = \beta(1-\alpha)\widehat{u}_{e,t} + (1-\beta)\alpha\widehat{v}_{e,t} + \frac{1}{\eta} D_{\mathrm{KL}}\big(\mathrm{Bern}(\beta) \| \, \mathrm{Bern}(\beta_t)\big), \tag{18}$$

$$D_{\mathrm{KL}}\big(\mathrm{Bern}(\beta) \| \, \mathrm{Bern}(\beta_t)\big) = \beta \log \frac{\beta}{\beta_t} + (1-\beta) \log \frac{1-\beta}{1-\beta_t}. \tag{19}$$

Let $d_t = (1-\alpha)\widehat{u}_{e,t} - \alpha\widehat{v}_{e,t}$. On the interior, the linear part of F_t is $\alpha\widehat{v}_{e,t} + \beta d_t$. Differentiate the two KL summands separately:

$$\frac{d}{d\beta}\left[\beta \log \frac{\beta}{\beta_t}\right] = \log \frac{\beta}{\beta_t} + 1, \tag{20}$$

$$\frac{d}{d\beta}\left[(1-\beta) \log \frac{1-\beta}{1-\beta_t}\right] = -\log \frac{1-\beta}{1-\beta_t} - 1. \tag{21}$$

The $+1$ and -1 cancel. Combining the logarithms and differentiating again gives

$$F_t'(\beta) = d_t + \frac{1}{\eta}\left[\log \frac{\beta}{1-\beta} - \log \frac{\beta_t}{1-\beta_t}\right], \tag{22}$$

$$F_t''(\beta) = \frac{1}{\eta\beta(1-\beta)} > 0. \tag{23}$$

The derivative diverges to $-\infty$ at zero and $+\infty$ at one, so there is a unique interior minimizer for finite d_t. Setting the first derivative to zero gives

$$\log \frac{1-\beta_{t+1}}{\beta_{t+1}} = \log \frac{1-\beta_t}{\beta_t} + \eta d_t, \tag{24}$$

$$\beta_{t+1} = \sigma\big(\mathrm{logit}(\beta_t) - \eta d_t\big), \qquad \sigma(z) = \frac{1}{1+\exp(-z)}. \tag{25}$$

Explicitly, $0 = \eta d_t + \mathrm{logit}(\beta_{t+1}) - \mathrm{logit}(\beta_t)$, where $\mathrm{logit}(b) = \log[b/(1-b)]$. Negating that equality gives the odds equation. Exponentiating the logit and solving $\beta/(1-\beta) = \exp(z)$ gives $\beta = \exp(z)/(1+\exp(z)) = \sigma(z)$. Because all other factors in Equation (13) are fixed, write $K_\alpha = (s_{e,0}/c_{e,0})\alpha/(1-\alpha) > 0$. Then

$$\log \lambda_{e,t} = \log K_\alpha + \log \frac{1-\beta_t}{\beta_t}, \tag{26}$$

$$\log \lambda_{e,t+1} - \log \lambda_{e,t} = \log \frac{1-\beta_{t+1}}{\beta_{t+1}} - \log \frac{1-\beta_t}{\beta_t} = \eta d_t. \tag{27}$$

Consequently,

$$\boxed{\log \lambda_{e,t+1} = \log \lambda_{e,t} + \eta\big[(1-\alpha)\widehat{u}_{e,t} - \alpha\widehat{v}_{e,t}\big].} \tag{28}$$

Equivalently, $\lambda_{e,t+1}/\lambda_{e,t} = \exp(\eta d_t)$. Success gain exceeding the weighted cost gain increases the cost penalty; cost gain exceeding the weighted success gain reduces it. The KL term here regularizes the *dual mixture*. It is separate from a reference-policy KL penalty in an RL loss; the current fixed-reward training loss has no such policy KL term.

2.7 Sampled feedback, calibration and projection

With n observations and a fixed baseline reference,

$$\widehat{s} = \frac{1}{n}\sum_{i=1}^{n} S_i, \qquad \widehat{c} = \frac{1}{n}\sum_{i=1}^{n} C_i, \tag{29}$$

$$\widehat{u} = \frac{\widehat{s} - s_0}{s_0}, \qquad \widehat{v} = \frac{c_0 - \widehat{c}}{c_0}, \qquad \widehat{d} = (1-\alpha)\frac{\widehat{s}}{s_0} + \alpha\frac{\widehat{c}}{c_0} - 1. \tag{30}$$

For independent identically distributed observations at a fixed policy, conditional on fixed reference values, set $a = (1-\alpha)/s_0$ and $b = \alpha/c_0$. Substitution yields $\widehat{d} = a\widehat{s} + b\widehat{c} - 1$. Adding a constant does not change variance, and expanding the square gives

$$\operatorname{Var}(a\widehat{s} + b\widehat{c}) = a^2 \operatorname{Var}(\widehat{s}) + b^2 \operatorname{Var}(\widehat{c}) + 2ab \operatorname{Cov}(\widehat{s}, \widehat{c}), \tag{31}$$

$$\operatorname{Var}(\widehat{s}) = \frac{1}{n^2}\sum_{i,j} \operatorname{Cov}(S_i, S_j) = \frac{\operatorname{Var}(S)}{n}, \tag{32}$$

$$\operatorname{Var}(\widehat{c}) = \frac{\operatorname{Var}(C)}{n}, \qquad \operatorname{Cov}(\widehat{s}, \widehat{c}) = \frac{1}{n^2}\sum_{i,j} \operatorname{Cov}(S_i, C_j) = \frac{\operatorname{Cov}(S, C)}{n}. \tag{33}$$

Cross-observation covariances vanish by independence; the within-observation success–cost covariance need not vanish. Substituting a, b yields

$$\operatorname{Var}(\widehat{d}) = \frac{1}{n}\Bigg[\frac{(1-\alpha)^2}{s_0^2}\operatorname{Var}(S) + \frac{\alpha^2}{c_0^2}\operatorname{Var}(C) + \frac{2\alpha(1-\alpha)}{s_0 c_0}\operatorname{Cov}(S, C)\Bigg]. \tag{34}$$

This expression explains the amplification from a small baseline solve rate. It excludes uncertainty in an estimated baseline and dependence from changing policies. In the implementation, accumulated observations can span several model updates; they describe a window of policies, not independent draws from one fixed current policy. An empirical baseline is frozen after calibration, but it remains an estimate of population performance. No claim of unbiased population ratios follows from that freeze.

The adaptive development protocols use training-only scale calibration

$$\lambda_{e,0} = \rho\frac{\widehat{s}_{e,0}}{\widehat{c}_{e,0}}, \qquad \rho = 0.1. \tag{35}$$

This sets $\lambda_{e,0}\widehat{c}_{e,0} = \rho\widehat{s}_{e,0}$; it is a cost scale choice. The original pilot instead used 0.01 success per token. The current fixed contrast uses 0 and 1/1024. These are different frozen experiments and are not interchangeable estimates of a frontier tangent. Zero baseline success or cost invalidates the relative controller and blocks qualification.

The implemented controller uses $\eta = 0.05$, accumulates at least 32 observations per effort, and applies the bounded log-space update

$$\log \lambda_{e,t+1} = \operatorname{clip}_{[\log 10^{-6},\, \log 1]}(\log \lambda_{e,t} + \eta\widehat{d}_t). \tag{36}$$

Batch rewards use the pre-update penalty; accumulated feedback sets the next penalty. Projection and accumulation are practical modifications to Equation (28). The current fixed contrast performs no controller updates.

2.8 Endpoint singularities, saturation and Pareto interpretation

The derivation assumes $0 < \alpha, \beta < 1$. At $\alpha = 1$ or $\beta = 0$, the scalarization divides by zero. At $\alpha = 0$ or $\beta = 1$ with the other denominator factors positive, its formula instead gives $\lambda = 0$, so the finite $\log \lambda$ update is undefined. Also, multiplying the common-improvement objective by $\alpha(1-\alpha)$ is no longer an equivalent transformation at either preference endpoint because that factor is zero. The original constrained problem still has well-defined endpoint meanings:

$$\alpha = 0: \quad \max_{\pi} v_e(\pi) \text{ subject to } u_e(\pi) \geq 0, \tag{37}$$

$$\alpha = 1: \quad \max_{\pi} u_e(\pi) \text{ subject to } v_e(\pi) \geq 0. \tag{38}$$

For $\alpha = 0$, the original inequalities become $u_e \geq 0$ and $\tau \leq v_e$; for $\alpha = 1$, they become $\tau \leq u_e$ and $v_e \geq 0$. Eliminating τ gives the two endpoint problems above. Extending the finite log-penalty feedback rule to those endpoints is a heuristic, not a consequence of the interior derivation.

If the true baseline is saturated, $s_{e,0} = 1$, then

$$u_e(\pi) = s_e(\pi) - 1 \leq 0, \qquad \min\{(1-\alpha)u_e(\pi), \alpha v_e(\pi)\} \leq 0. \tag{39}$$

The base policy attains zero. Every accuracy-preserving cost reduction also attains zero because $u_e = 0$ and $v_e > 0$. Thus the interior max-min objective does not distinguish those reductions from the base policy. A noninferiority-constrained cost optimizer would be a separately specified objective.

For a differentiable attainable frontier $s = f(c)$, a hypothetical interior local optimum of $f(c) - \lambda c$ must satisfy

$$f'(c^*) = \lambda. \tag{40}$$

This follows by differentiating the scalar objective: $d[f(c) - \lambda c]/dc = f'(c) - \lambda$ and setting it to zero at an unconstrained interior optimum. A strict local maximum additionally requires appropriate curvature; $f''(c^*) < 0$ is a sufficient second-order condition when f is twice differentiable. At an attainable-set boundary the stationary equation need not hold. This is a local necessary condition, not an initialization estimator or global guarantee. A global maximizer with $\lambda > 0$ cannot be dominated in both success and cost: any policy with $s' \geq s^*$, $c' \leq c^*$ and at least one strict inequality would have strictly larger scalar utility. Nonconvex attainable sets can nevertheless contain nondominated points that no linear penalty recovers. The LoRA policy class is nonconvex; the present experiment neither measures an entire frontier nor establishes convergence of alternating stochastic policy and controller updates.

3 Task, policy and training

The historical exploratory configuration and the new mechanism study use pinned Qwen3-4B-Instruct-2507 weights and rank-eight LoRA over attention q/k/v/o projections. The model receives candidate expression repairs and visible input/output constraints. The restricted evaluator checks hidden cases; generated Python is never executed. Constrained answer formatting is not evidence of learned repository-level repair.

More explicitly, task x supplies four predefined candidate functions $f_{x,a}$, a target function f_x^*, and a nonempty hidden-input set $\mathcal{H}_x$. If the sampled answer selects candidate a, the scoring rule is

$$S(x,(z,a)) = \prod_{\xi \in \mathcal{H}_x} \mathbf{1}\{f_{x,a}(\xi) = f_x^*(\xi)\}, \qquad P(x,a) = \frac{1}{|\mathcal{H}_x|} \sum_{\xi \in \mathcal{H}_x} \mathbf{1}\{f_{x,a}(\xi) = f_x^*(\xi)\}. \tag{41}$$

The product is one precisely when every hidden case agrees. P is the logged fraction of passing cases; the reward uses S, not P. Candidate expressions are interpreted in the bounded permitted expression language. Parsing or evaluation failure produces failure. The scratch reasoning can influence the selected answer and its cost, but the scorer judges the selected candidate rather than the truth of the scratch prose.

3.1 Stopped trajectory distribution

Write a trajectory as $Y = (Z, A)$, with scratch tokens $Z = (z_1, \ldots, z_L)$ and $A \in \mathcal{A} = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}\}$. Let $K = 2048$ and

$$L = \min\{T_{\mathrm{EOS}}, K\}, \qquad C(Y) = L + 1, \qquad 1 \le C(Y) \le K + 1. \tag{42}$$

Here T_{EOS} is the first sampled EOS position, or infinity if none is sampled before the cap. Natural termination includes the EOS action and its likelihood. Reaching the cap does not add a sampled stop action. The answer contributes one generated token. A cap hit is

$$H(Y) = \mathbf{1}\{L = K,\ z_K \neq \mathrm{EOS}\}. \tag{43}$$

Input tokens, wall time and dollars are reported separately from C.

For temperature $T = 0.7$ and model logits ℓ_θ, scratch probabilities are

$$p_{\theta,T}(z_t \mid x, z_{<t}) = \frac{\exp(\ell_\theta(z_t; x, z_{<t})/T)}{\sum_{v \in \mathcal{V}} \exp(\ell_\theta(v; x, z_{<t})/T)}. \tag{44}$$

Let $r(x, z)$ be the deterministic decode, message-construction and re-encoding operation used to build the readout prompt. Its restricted answer distribution is

$$p^{\mathcal{A}}_{\theta,T}(a \mid r(x, z)) = \frac{\exp(\ell_\theta(a; r(x, z))/T)}{\sum_{b \in \mathcal{A}} \exp(\ell_\theta(b; r(x, z))/T)}. \tag{45}$$

The likelihood of a valid stopped trajectory is

$$q_\theta(z, a \mid x) = \prod_{t=1}^{L} p_{\theta,T}(z_t \mid x, z_{<t})\, p^{\mathcal{A}}_{\theta,T}(a \mid r(x, z)), \tag{46}$$

$$\log q_\theta(z, a \mid x) = \sum_{t=1}^{L} \log p_{\theta,T}(z_t \mid x, z_{<t}) + \log p^{\mathcal{A}}_{\theta,T}(a \mid r(x, z)). \tag{47}$$

This product is the conditional-probability chain rule. At each scratch position, the next token is drawn conditional on all previous tokens. A valid sequence either ends at its first EOS or reaches K without an earlier EOS. Thus the stop rule selects leaves of a finite probability tree; no independent probability factor for "deciding to stop" is added. The probabilities of those leaves sum to one because all outgoing token probabilities sum to one at each internal node. The conditional answer probabilities also sum to one, so $\sum_{z,a} q_\theta(z, a \mid x) = 1$. Taking the logarithm converts the product into the sum in Equation (47). The softmax normalization is part of the score. For a sampled scratch token,

$$\nabla_\theta \log p_{\theta,T}(z \mid h) = \frac{1}{T}\left[\nabla_\theta \ell_\theta(z; h) - \sum_v p_{\theta,T}(v \mid h) \nabla_\theta \ell_\theta(v; h)\right]. \tag{48}$$

For completeness, define $Z_\theta(h) = \sum_v \exp(\ell_\theta(v;h)/T)$. Holding the recorded token context h fixed,

$$\log p_{\theta,T}(z \mid h) = \ell_\theta(z;h)/T - \log Z_\theta(h), \tag{49}$$

$$\nabla_\theta \log Z_\theta(h) = \frac{1}{Z_\theta(h)} \sum_v \exp(\ell_\theta(v;h)/T) \frac{\nabla_\theta \ell_\theta(v;h)}{T} \tag{50}$$

$$= \frac{1}{T} \sum_v p_{\theta,T}(v \mid h) \nabla_\theta \ell_\theta(v;h). \tag{51}$$

Subtracting this normalizer gradient from the selected-logit gradient produces the preceding score identity. Here h means fixed input tokens; internal model activations are still differentiated. For the answer, the same identity holds with the sum restricted to $\mathcal{A}$. No gradient is taken through the sampled token IDs or discrete evaluator. Both the scratch likelihood and the separate readout likelihood contribute parameter gradients; host-provided prompts contribute no sampled-action likelihood. Prompt processing still influences parameter gradients through the model computation.

3.2 Score-function policy gradient

Fix x, θ, λ during a sampling group. Define

$$J_{\lambda,x}(\theta) = \sum_y q_\theta(y \mid x) R_\lambda(x,y), \qquad h_\theta(y,x) = \nabla_\theta \log q_\theta(y \mid x). \tag{52}$$

This is the score-function principle underlying REINFORCE [5]. The finite token vocabulary and fixed ceiling define a finite trajectory space; its support does not change with θ under the qualified sampling rule. Assuming finite logits and differentiability, differentiation and summation commute. Since S and C are fixed functions of a realized discrete trajectory, the logarithm derivative supplies the essential identity

$$\nabla_\theta \log q_\theta(y \mid x) = \frac{\nabla_\theta q_\theta(y \mid x)}{q_\theta(y \mid x)}, \qquad \nabla_\theta q_\theta(y \mid x) = q_\theta(y \mid x) h_\theta(y,x). \tag{53}$$

All valid trajectories have positive probability under finite-logit softmax. The product rule has no $q_\theta \nabla_\theta R$ term because the realized discrete reward has no direct parameter dependence at fixed λ. Therefore,

$$\nabla_\theta J_{\lambda,x}(\theta) = \sum_y R_\lambda(x,y) \nabla_\theta q_\theta(y \mid x) \tag{54}$$

$$= \mathbb{E}_{Y \sim q_\theta}[R_\lambda(x,Y) h_\theta(Y,x)]. \tag{55}$$

$$\mathbb{E}[h_\theta(Y,x)] = \sum_y \nabla_\theta q_\theta(y \mid x) = \nabla_\theta 1 = 0. \tag{56}$$

Averaging Equation (55) over an independently sampled task distribution gives the corresponding population objective. The implemented study instead uses a frozen, shuffled finite task schedule; the conditional estimator remains valid at its selected task, without making every update an independent population-task draw.

3.3 Leave-one-out estimator and proof

For $G = 4$ independent trajectories at the same fixed task, policy and penalty, define

$$b_{-i} = \frac{1}{G-1} \sum_{j \neq i} R_j, \qquad A_i = R_i - b_{-i}, \qquad \widehat{g} = \frac{1}{G} \sum_{i=1}^{G} A_i h_\theta(Y_i, x). \tag{57}$$

Proposition 3.1 (Conditional unbiasedness of the leave-one-out score estimate)**.** *Under the sampling and differentiability assumptions above,* $\mathbb{E}[\widehat{g} \mid x, \theta, \lambda] = \nabla_\theta J_{\lambda,x}(\theta)$.

Proof. For each i, b_{-i} depends only on other independent trajectories. Using Equation (56),

$$\mathbb{E}[b_{-i} h_\theta(Y_i, x) \mid x, \theta, \lambda] = \mathbb{E}[b_{-i} \mid x, \theta, \lambda]\, \mathbb{E}[h_\theta(Y_i, x) \mid x, \theta, \lambda] = 0, \tag{58}$$

$$\mathbb{E}[A_i h_\theta(Y_i, x) \mid x, \theta, \lambda] = \mathbb{E}[R_i h_\theta(Y_i, x) \mid x, \theta, \lambda] = \nabla_\theta J_{\lambda,x}(\theta). \tag{59}$$

Averaging over i proves the claim. The baseline samples are random but conditionally independent of the trajectory whose score they multiply. □

Let $\overline{R} = G^{-1} \sum_i R_i$ and $\overline{h} = G^{-1} \sum_i h_i$. Algebra gives

$$b_{-i} = \frac{\sum_j R_j - R_i}{G-1} = \frac{G\overline{R} - R_i}{G-1}, \tag{60}$$

$$A_i = \frac{(G-1)R_i - G\overline{R} + R_i}{G-1} = \frac{G}{G-1}(R_i - \overline{R}), \tag{61}$$

$$\sum_i (R_i - \overline{R}) = \sum_i R_i - G\overline{R} = 0, \tag{62}$$

$$\widehat{g} = \frac{1}{G-1} \sum_i (R_i - \overline{R}) h_i, \tag{63}$$

$$\sum_i (R_i - \overline{R})\overline{h} = \overline{h} \sum_i (R_i - \overline{R}) = 0. \tag{64}$$

Subtracting the final zero term from the score estimate gives

$$A_i = \frac{G}{G-1}(R_i - \overline{R}), \qquad \sum_i A_i = 0, \tag{65}$$

$$\widehat{g} = \frac{1}{G-1} \sum_i (R_i - \overline{R})(h_i - \overline{h}). \tag{66}$$

The estimator is the sample covariance between reward and score. Subtracting the inclusive group mean without the $G/(G-1)$ correction would shrink its expectation by $(G-1)/G$. There is no reward-standard-deviation normalization, output-length normalization, replay epoch or second optimizer step over the sampled group.

The differentiated surrogate is

$$\mathcal{L}(\theta; Y_{1:G}) = -\frac{1}{G} \sum_i \mathrm{stopgrad}(A_i) \log q_\theta(Y_i \mid x). \tag{67}$$

The realized advantages are held constant during backpropagation. At the policy that generated the group, $-\nabla_\theta \mathcal{L} = \widehat{g}$. All four samples are drawn before the parameter update. If the penalty is adaptive, it is still fixed during this step; differentiating through the controller would define a different algorithm.

The loss-to-update sign can be checked directly:

$$\nabla_\theta \mathcal{L} = -\frac{1}{G} \sum_i A_i \nabla_\theta \log q_\theta(Y_i \mid x) = -\widehat{g}. \tag{68}$$

Plain gradient descent on this loss would give $\theta^+ = \theta - \gamma \nabla_\theta \mathcal{L} = \theta + \gamma \widehat{g}$, a reward-ascent step. Adam later modifies this direction using moments. The positive/negative advantages are realized numerical coefficients; they are not derivatives of the hidden scorer. We write a_i^{ans} for a particular sampled answer letter when distinguishing it from the advantage A_i.

3.4 How the token penalty changes the gradient signal

Let $\overline{S}_{-i}$ and $\overline{C}_{-i}$ be leave-one-out means. Then

$$A_i(\lambda) = (S_i - \overline{S}_{-i}) - \lambda(C_i - \overline{C}_{-i}), \tag{69}$$

$$\widehat{g}_\lambda = \widehat{g}_S - \lambda \widehat{g}_C, \tag{70}$$

$$\nabla_\theta J_{\lambda,x} = \nabla_\theta s_x - \lambda \nabla_\theta c_x. \tag{71}$$

Success-only training has no reward variation in an all-correct or all-incorrect group. The token penalty can create a nonzero advantage in those same groups when generated lengths vary. If both success and cost are tied, both utilities have zero group signal. The decomposition is linear at the gradient level. Gradient clipping and adaptive optimizer moments generally make actual parameter updates nonlinear in λ, so a gradient difference does not itself establish different trained behavior.

The current fixed contrast uses $\lambda \in \{0, 1/1024\}$, 64 groups and seed 17. It produces $64 \times 4 = 256$ fresh trajectories per arm. The stronger cost scale was chosen after development diagnostics; this makes the comparison exploratory.

3.5 From a gradient estimate to local progress: an idealized bound

The previous proof identifies an expected gradient, not the final trained solution. The connection to local progress can be made precise for an ideal plain-ascent update, distinct from the implemented clipped Adam step. Fix the task and penalty, write $J = J_{\lambda,x}$, and suppose J is twice differentiable with Hessian operator norm at most L_H along every relevant update segment. For $\theta^+ = \theta + \gamma\widehat{g}$, Taylor's theorem and that bound give

$$\begin{aligned} J(\theta + \gamma\widehat{g}) &= J(\theta) + \gamma \nabla J(\theta)^\top \widehat{g} \\ &\quad + \frac{\gamma^2}{2} \widehat{g}^\top \nabla^2 J(\theta + \xi\gamma\widehat{g})\widehat{g}, \quad 0 < \xi < 1, \end{aligned} \tag{72}$$

$$J(\theta + \gamma\widehat{g}) \geq J(\theta) + \gamma \nabla J(\theta)^\top \widehat{g} - \frac{L_H \gamma^2}{2} \|\widehat{g}\|_2^2. \tag{73}$$

Taking the conditional expectation at the fixed x, θ, λ and using the unbiasedness proposition yields

$$\mathbb{E}[J(\theta^+)] - J(\theta) \geq \gamma \|\nabla J(\theta)\|_2^2 - \frac{L_H \gamma^2}{2} \mathbb{E}\|\widehat{g}\|_2^2. \tag{74}$$

For $\sigma_g^2 = \mathbb{E}\|\widehat{g} - \nabla J(\theta)\|_2^2$, expanding the square and using $\mathbb{E}[\widehat{g} - \nabla J] = 0$ gives $\mathbb{E}\|\widehat{g}\|_2^2 = \|\nabla J\|_2^2 + \sigma_g^2$. When the gradient is nonzero and $L_H > 0$, the bound is positive for

$$0 < \gamma < \frac{2\|\nabla J\|_2^2}{L_H(\|\nabla J\|_2^2 + \sigma_g^2)}. \tag{75}$$

This conditional calculation explains why gradient signal, estimator noise and step size all matter. It does not give an implementable certified step size here: L_H and the population gradient/noise are not measured. It neither establishes a global optimum nor applies unchanged to clipping, Adam, a changing task or an adaptive penalty. No monotonic improvement is asserted for the actual run.

3.6 Worked training-group example (illustrative, not measured)

Suppose four sampled repairs have $S = (1, 1, 0, 0)$ and $C = (512, 1024, 512, 1024)$. Under success-only reward, $R = S$, $\overline{R} = 1/2$, and $A_i = \frac{4}{3}(R_i - \overline{R})$ gives

$$A^{\text{succ}} = (2/3, 2/3, -2/3, -2/3). \tag{76}$$

For $\lambda = 1/1024$, the token charges are $(1/2, 1, 1/2, 1)$, so

$$R^{\text{pen}} = (1/2, 0, -1/2, -1), \qquad \overline{R}^{\text{pen}} = -1/4, \tag{77}$$

$$A^{\text{pen}} = (1, 1/3, -1/3, -1). \tag{78}$$

For example, $b_{-1} = (0 - 1/2 - 1)/3 = -1/2$, and $A_1 = 1/2 - (-1/2) = 1$. Both advantage vectors sum to zero. If $Q_i = \log q_\theta(Y_i \mid x)$ denotes the complete scratch-plus-answer log likelihood, the penalty loss for these saved samples is

$$\mathcal{L}_{\text{pen}} = -\frac{1}{4}\left(Q_1 + \frac{1}{3}Q_2 - \frac{1}{3}Q_3 - Q_4\right). \tag{79}$$

No numerical parameter update can be inferred from these rewards alone: its value also requires each $\nabla_\theta Q_i$, the current adapter, and the optimizer state. This example illustrates exactly how shorter trajectories change their credit at the same success value.

As a separate controller example, suppose a fixed baseline has $s_0 = 0.8$, $c_0 = 1000$, and a feedback window estimates $\widehat{s} = 0.84$, $\widehat{c} = 900$. Then $\widehat{u} = 0.04/0.8 = 0.05$, $\widehat{v} = 100/1000 = 0.10$. With $\alpha = 1/2$ and $\eta = 0.05$, $\widehat{d} = 0.5(0.05) - 0.5(0.10) = -0.025$ and

$$\lambda^+ = \lambda \exp(-0.00125) \simeq 0.99875\lambda. \tag{80}$$

This unprojected update slightly reduces the penalty because weighted cost gain exceeds weighted success gain. It is an illustrative adaptive calculation, not a controller update performed by the current fixed-penalty run.

3.7 LoRA parameterization and optimizer

Following the low-rank parameterization of Hu et al. [6], for a frozen projection $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$,

$$W(\theta) = W_0 + \kappa BA, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad r = 8, \quad \kappa = 16/8 = 2. \tag{81}$$

The trainable count per projection is $r(d_{\text{in}} + d_{\text{out}})$ rather than $d_{\text{in}} d_{\text{out}}$. If $G_W = \partial \mathcal{L}/\partial W$, the matrix chain rule gives

$$\frac{\partial \mathcal{L}}{\partial A} = \kappa B^\top G_W, \qquad \frac{\partial \mathcal{L}}{\partial B} = \kappa G_W A^\top. \tag{82}$$

One can verify both dimensions and the chain rule by perturbing the matrices. Use the Frobenius inner product $\langle U, V \rangle_F = \operatorname{tr}(U^\top V) = \sum_{ij} U_{ij} V_{ij}$. To first order,

$$dW = \kappa(dB\, A + B\, dA), \tag{83}$$

$$d\mathcal{L} = \langle G_W, dW \rangle_F \tag{84}$$

$$= \kappa \operatorname{tr}(G_W^\top dB\, A) + \kappa \operatorname{tr}(G_W^\top B\, dA) \tag{85}$$

$$= \langle \kappa G_W A^\top, dB \rangle_F + \langle \kappa B^\top G_W, dA \rangle_F. \tag{86}$$

The coefficients of dB and dA are their gradients. Thus $G_W A^\top$ has shape $d_{\text{out}} \times r$ and $B^\top G_W$ has shape $r \times d_{\text{in}}$, matching their parameters. Because W_0 is frozen, it receives no optimizer update even though the computation uses it. For zero-initialized B, the initial effective weight equals W_0 and the first A gradient is zero, while the B gradient can be nonzero. All arms within each study start from the same archived adapter. LoRA and attention dropout are zero; the CUDA backward pass enables activation checkpointing without adding stochastic dropout.

Let $g_t = \nabla_\theta \mathcal{L}_t$ evaluated at θ_{t-1} and clip at unit norm:

$$\widetilde{g}_t = g_t \min\{1, 1/\|g_t\|_2\}, \qquad \widetilde{g}_t = 0 \text{ if } g_t = 0. \tag{87}$$

With t counting actual optimizer updates, zero-weight-decay AdamW reduces to the Adam recurrence [7]:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\widetilde{g}_t, \tag{88}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\widetilde{g}_t^{\odot 2}, \tag{89}$$

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{90}$$

$$\theta_t = \theta_{t-1} - \gamma \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}. \tag{91}$$

Operations on vectors are elementwise; $\gamma = 10^{-5}$, $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\varepsilon = 10^{-8}$. The optimizer is skipped entirely for an exactly tied reward group: parameters, moments and optimizer step counters do not advance. Calling Adam with a zero gradient could still move weights through previous momentum, so zero-advantage skipping is part of the specified algorithm. The number of optimizer updates can be below 64. Unbiasedness of Equation (57) does not imply unbiasedness after clipping, Adam, finite precision or changing penalties.

Here clipping multiplies every gradient coordinate by the same positive scalar. If $\|g_t\|_2 > 1$, its resulting norm is $\|g_t\|_2/\|g_t\|_2 = 1$; otherwise the gradient is retained. Adam's recurrences and bias corrections define the implemented optimizer; they are not solutions of the constrained objective. Their interpretation follows by unrolling $m_0 = v_0 = 0$:

$$m_t = (1-\beta_1)\sum_{j=1}^{t} \beta_1^{t-j}\widetilde{g}_j, \tag{92}$$

$$v_t = (1-\beta_2)\sum_{j=1}^{t} \beta_2^{t-j}\widetilde{g}_j^{\odot 2}, \tag{93}$$

$$(1-\beta)\sum_{j=1}^{t} \beta^{t-j} = 1-\beta^t. \tag{94}$$

For a hypothetical sequence with constant first moment $\mathbb{E}[\widetilde{g}_j] = \mu$ and constant second moment $\mathbb{E}[\widetilde{g}_j^{\odot 2}] = \nu$, these equalities give $\mathbb{E}[m_t] = (1-\beta_1^t)\mu$ and $\mathbb{E}[v_t] = (1-\beta_2^t)\nu$. Division removes the initial-zero attenuation under those assumptions. Actual training gradients are changing and clipped; this calculation does not assert stationarity or unbiasedness of the ratio $\widehat{m}_t/(\sqrt{\widehat{v}_t} + \varepsilon)$.

In execution order, a group selects its frozen task, draws all four trajectories at the unchanged current parameters, evaluates their success and cost, computes detached rewards/advantages, reconstructs and differentiates both likelihood components, checks replay errors and finiteness, clips the loss gradient, and takes one Adam step unless all advantages are exactly zero. The completed group counter advances even for a skipped update. Numerical validation and durable recovery protect the resulting records before the next group. Final evaluation begins only after all planned groups complete; its comparisons are not gradient-training inputs.

3.8 Numerical replay and loss reconstruction gates

Write $\ell_{i,t}^{\text{sample}}$ and $\ell_{i,t}^{\text{train}}$ for sampled and differentiated scratch log probabilities. Each contributing trajectory must satisfy

$$\max_t |\ell_{i,t}^{\text{sample}} - \ell_{i,t}^{\text{train}}| \le 0.10, \tag{95}$$

$$\frac{1}{L_i}\sum_t |\ell_{i,t}^{\text{sample}} - \ell_{i,t}^{\text{train}}| \le 0.02, \tag{96}$$

$$|\ell_{i,A}^{\text{sample}} - \ell_{i,A}^{\text{train}}| \le 0.02, \tag{97}$$

with errors in nats and zero scratch error for an empty scratch sequence. Define recorded loss components

$$\ell_i = -\frac{A_i}{4}\left(\sum_t \ell_{i,t}^{\text{train}} + \ell_{i,A}^{\text{train}}\right). \tag{98}$$

The recorded loss must reconstruct as

$$\left|\mathcal{L}_{\text{recorded}} - \sum_i \ell_i\right| \le 10^{-4} + 10^{-5}\sum_i |\ell_i|. \tag{99}$$

Nonfinite losses, gradients or saved states fail the workload. Zero measured final parameter change remains a valid negative learning outcome. Likelihood agreement and loss reconstruction test numerical consistency; they are not proofs of generalization or of every gradient's exact mathematical value. Gates and tolerances remain fixed after observing outcomes.

4 Evaluation and evidence

The historical exploratory evaluation fixes $N = 32$ development tasks and $M = 2$ matched samples per task. It evaluates only the final adapter, with no intermediate performance-based checkpoint selection. The base and completed success-only evaluation are reused from the audited historical recovery. The newly trained token-penalty arm completed its final matched development evaluation; all three sets of 64 records were reconstructed by the independent audit. No sealed final-test task was used in that historical comparison. Section 7 reports the separate 96-task final evaluation and its crossed seed/task analysis.

4.1 Paired task means and utility contrasts

For policy a, task j and sample m, retain $S_{a,jm}, C_{a,jm}, H_{a,jm}$. For any metric $Y \in \{S, C, H\}$, compute

$$\overline{Y}_{a,j} = \frac{1}{M}\sum_{m=1}^{M} Y_{a,jm}, \qquad \widehat{\mu}_{a,Y} = \frac{1}{N}\sum_{j=1}^{N} \overline{Y}_{a,j}, \tag{100}$$

$$d_{j,Y} = \overline{Y}_{\text{pen},j} - \overline{Y}_{\text{succ},j}, \qquad \widehat{\Delta}_Y = \frac{1}{N}\sum_j d_{j,Y}. \tag{101}$$

Negative $\widehat{\Delta}_C$ indicates fewer generated tokens. Negative $\widehat{\Delta}_S$ indicates lower success probability. Relative cost change is a ratio of *overall means*, not an average of task-specific ratios:

$$\widehat{\delta}_C = \frac{\widehat{\mu}_{\text{pen},C}}{\widehat{\mu}_{\text{succ},C}} - 1. \tag{102}$$

Because each task has the same M samples, the task-weighted mean also equals the completion mean:

$$\widehat{\mu}_{a,Y} = \frac{1}{N}\sum_{j}\left(\frac{1}{M}\sum_{m} Y_{a,jm}\right) = \frac{1}{NM}\sum_{j,m} Y_{a,jm}. \tag{103}$$

Success is binary, so dividing the successful-completion count by $NM = 64$ yields the reported solve rate. For cost, $\widehat{\delta}_C = \widehat{\Delta}_C/\widehat{\mu}_{\mathrm{succ},C}$, requiring a positive denominator. A value -0.10 therefore means ten percent fewer tokens relative to the control mean, whereas $\widehat{\Delta}_S = -0.05$ means five percentage points lower solve probability, not five percent relative accuracy loss. The mean utility of either policy is evaluated under both frozen reward functions:

$$\widehat{J}_\lambda(a) = \widehat{\mu}_{a,S} - \lambda\widehat{\mu}_{a,C}, \qquad \widehat{\Delta}_{J_\lambda} = \widehat{\Delta}_S - \lambda\widehat{\Delta}_C, \quad \lambda \in \{0, 1/1024\}. \tag{104}$$

Higher utility under a specified penalty can coexist with lower accuracy; it does not establish noninferiority. Shared task and sampling-seed identities make the comparison paired. They do not force equal trajectories once policies differ. The variance identity

$$\mathrm{Var}(\widehat{\Delta}_Y) = \mathrm{Var}(\widehat{\mu}_{\mathrm{pen},Y}) + \mathrm{Var}(\widehat{\mu}_{\mathrm{succ},Y}) - 2\,\mathrm{Cov}(\widehat{\mu}_{\mathrm{pen},Y}, \widehat{\mu}_{\mathrm{succ},Y}) \tag{105}$$

shows why positive covariance can reduce variance through pairing; positive covariance is not guaranteed. To derive it, let $U = \widehat{\mu}_{\mathrm{pen},Y}$ and $V = \widehat{\mu}_{\mathrm{succ},Y}$. Since $\widehat{\Delta}_Y = U - V$, expand $\mathbb{E}[((U - \mathbb{E}U) - (V - \mathbb{E}V))^2]$. The two squares give $\mathrm{Var}(U)$ and $\mathrm{Var}(V)$, and the cross term gives $-2\,\mathrm{Cov}(U, V)$.

4.2 Family-stratified task-cluster bootstrap

Partition tasks into four families $\mathcal{T}_f$, with $n_f = |\mathcal{T}_f|$ and $\sum_f n_f = N$. The family contrast and overall weighted contrast are

$$\widehat{\Delta}_{f,Y} = \frac{1}{n_f}\sum_{j\in\mathcal{T}_f} d_{j,Y}, \qquad \widehat{\Delta}_Y = \sum_f \frac{n_f}{N}\widehat{\Delta}_{f,Y}. \tag{106}$$

For each replicate b, sample n_f task IDs with replacement within each family:

$$I^{(b)}_{f,k} \overset{\text{iid}}{\sim} \mathrm{Unif}(\mathcal{T}_f), \qquad \widehat{\Delta}^{(b)}_Y = \frac{1}{N}\sum_f \sum_{k=1}^{n_f} d_{I^{(b)}_{f,k},Y}. \tag{107}$$

One resampled task ID carries its two samples and all policy arms together. This preserves the paired structure and does not treat the 64 completions as 64 independent tasks. Relative cost is recomputed from the resampled arm means in each replicate.

In particular, retain repeated IDs as repeated contributions, rather than deduplicating them. The resampled means and ratio are

$$\widehat{\mu}^{(b)}_{a,Y} = \frac{1}{N}\sum_f \sum_{k=1}^{n_f} \overline{Y}_{a,I^{(b)}_{f,k}}, \tag{108}$$

$$\widehat{\delta}^{(b)}_C = \frac{\widehat{\mu}^{(b)}_{\mathrm{pen},C}}{\widehat{\mu}^{(b)}_{\mathrm{succ},C}} - 1. \tag{109}$$

Conditional on the observed task means, every original task in family f has probability $1/n_f$ on each of its n_f draws. Thus

$$\mathbb{E}_*[\widehat{\Delta}_Y^{(b)}] = \frac{1}{N}\sum_f n_f \left(\frac{1}{n_f}\sum_{j\in\mathcal{T}_f} d_{j,Y}\right) = \widehat{\Delta}_Y, \tag{110}$$

where $\mathbb{E}_*$ averages only over bootstrap resampling of the fixed data. This centering identity holds for the linear contrast; it does not imply $\mathbb{E}_*[\widehat{\delta}_C^{(b)}] = \widehat{\delta}_C$ for a nonlinear ratio or establish population coverage.

The implementation fixes $B = 10{,}000$ replicates and bootstrap seed 7501. Sort a contrast's draws as $z_{(0)} \le \cdots \le z_{(B-1)}$ and use the exact recorded order-statistic interval

$$\left[z_{(\lfloor 0.025(B-1)\rfloor)},\quad z_{(\lceil 0.975(B-1)\rceil)}\right]. \tag{111}$$

These are descriptive task intervals for a selected development set. Repeated model, prompt and reward design used that set. The intervals do not undo selection, estimate variation across independent training seeds, or separately resample repeated decoding for a fixed task. For $B = 10{,}000$, the endpoint indices are $\lfloor 249.975\rfloor = 249$ and $\lceil 9749.025\rceil = 9750$ in zero-based indexing. Sorting and selecting these observed draws specifies the interval algorithm; a nominal 95% percentile interval is not an exact finite-sample 95% coverage theorem.

Let R denote training randomness in a hypothetical repeated study. The law of total variance gives

$$\mathrm{Var}(\widehat{\Delta}) = \mathbb{E}_R[\mathrm{Var}(\widehat{\Delta} \mid R)] + \mathrm{Var}_R(\mathbb{E}[\widehat{\Delta} \mid R]). \tag{112}$$

A single realized training seed cannot estimate the second term. Boundary observations can also produce collapsed bootstrap intervals: empirical perfect success does not imply zero population failure probability. Prospective confirmation requires untouched tasks and multiple independent paired training seeds.

The variance decomposition follows without assuming normality. Set $Z = \widehat{\Delta}$, $\mu = \mathbb{E}Z$, and $\mu_R = \mathbb{E}[Z \mid R]$. Write

$$Z - \mu = (Z - \mu_R) + (\mu_R - \mu). \tag{113}$$

After squaring and taking expectations, the cross term is zero because

$$\mathbb{E}[(Z - \mu_R)(\mu_R - \mu)] = \mathbb{E}_R[(\mu_R - \mu)\mathbb{E}[Z - \mu_R \mid R]] = 0. \tag{114}$$

The first squared term is $\mathbb{E}_R[\mathrm{Var}(Z \mid R)]$ and the second is $\mathrm{Var}_R(\mu_R)$, yielding the displayed law. Resampling tasks within one fitted pair does not generate new fitted pairs and therefore cannot recover the second term.

4.3 Decision boundaries and evidence reconstruction

The frozen exploratory discussion trigger asks for at least 10% lower mean cost, at most five percentage points lower solve rate, and a wholly negative descriptive cost interval:

$$\widehat{\delta}_C \le -0.10, \qquad \widehat{\Delta}_S \ge -0.05, \qquad \mathrm{upper}(\mathrm{CI}_{\Delta_C}) < 0. \tag{115}$$

This is grounds for discussing a separately approved study. It is not an automatic launch, a significance-adjusted search criterion or a test establishing success noninferiority. A prospective

noninferiority claim would require a predeclared margin and valid uncertainty for the success difference, with the lower bound above the negative margin under the specified analysis.

Independent audits recompute S, C, R_i, A_i, replay discrepancies, Equation (99), adapter changes and all paired contrasts from frozen source and complete records. They validate completeness and provenance before analysis. Partial arms or qualification trajectories do not substitute for the planned final evaluation. Section B specifies the exact state and off-Pod evidence requirements.

5 Historical exploratory results

Observation	Retained finding	Scope
Earlier Qwen3 fixed/adaptive CUDA pilot	Identical sampled outcomes: 32/32 reasoning answers correct, 735.66 mean tokens in both arms	One-seed development pilot; no adaptive advantage detected
Compact reward diagnostic	1/8 mixed-success groups met the joint sensitivity threshold; 4 required	Failed frozen qualification; conditional training did not launch
Initial adapter in protected fixed-reward retry	53/64 successes, 1,554.83 mean tokens	Reused development evaluation
Complete success-only arm in protected retry	52/64 successes, 1,556.69 mean tokens; 64 groups, 30 optimizer updates	Audited reused control in the completed continuation
Token-penalty arm in protected retry	Seven groups and seven updates survived; no trained checkpoint	Incomplete arm; no final evaluation
Completed token-penalty continuation	54/64 successes, 1,567.89 mean tokens; 64 groups, 59 optimizer updates	Independently audited exploratory result; reused control

The first two observations use different historical protocols and are not pooled with the current comparison. The one-answer difference between the recovered initial and success-only evaluations does not establish improvement or degradation. The completed continuation provides the missing penalty arm; its final point estimate uses slightly more tokens. Both principal descriptive intervals include zero. Under the unchanged practical review rule (at least 10 percent cost reduction with an entirely negative cost interval for a promising signal), this is a small or uncertain response. It does not qualify for an automatic follow-up or justify extending the horizon.

5.1 Completed fixed-reward contrast

Table 1 reports only the matched final development evaluation. Each policy has 64 completions grouped into 32 paired tasks. No best checkpoint was selected.

Policy	Correct/64	Solve rate	Mean tokens	Ceiling hits/64
Initial adapter	53	82.8125%	1,554.828125	21
Success-only	52	81.2500%	1,556.687500	20
Token penalty	54	84.3750%	1,567.890625	23

For the primary cost contrast and mandatory accuracy companion, every arithmetic step is

$$\widehat{\Delta c} = 1567.890625 - 1556.687500 = 11.203125 \text{ tokens}, \tag{116}$$

$$100\widehat{\Delta s} = 100(54/64 - 52/64) = 100(2/64) = 3.125 \text{ percentage points}, \tag{117}$$

$$100\widehat{r}_c = 100\left(\frac{1567.890625}{1556.687500} - 1\right) = 0.719677199\ldots\,\%. \tag{118}$$

The unchanged family-stratified, paired task-cluster bootstrap (10,000 replicates, seed 7501) yields descriptive 95% intervals of $[-65.09375, 88.34375]$ tokens, $[-6.25, 14.0625]$ accuracy percentage points and $[-4.06534, 5.88747]\%$ relative cost change. These intervals condition on the trained models and inspected development split; they exclude training-seed and design-selection uncertainty.

Under the penalty utility, the two final development point estimates are

$$\widehat{J}_{\text{succ}} = 0.8125 - 1556.6875/1024 = -0.7077026367\ldots, \tag{119}$$

$$\widehat{J}_{\text{pen}} = 0.84375 - 1567.890625/1024 = -0.6873931885\ldots, \tag{120}$$

$$\widehat{\Delta J} = \widehat{\Delta s} - \widehat{\Delta c}/1024 = 0.03125 - 11.203125/1024 = 0.0203094482\ldots. \tag{121}$$

This utility point estimate combines slightly more correct answers with slightly longer traces; it is not evidence of an efficiency gain or adaptive superiority. The penalty arm makes 59 actual optimizer updates versus 30 in the control because tied rewards skip updates. Both receive the same 64-group, 256-trajectory exposure. Audited adapter-change norms are 0.20668 and 0.15222 respectively; nonzero parameter movement does not imply a useful behavioral response. The four family cost differences have mixed signs (62.06, −48.63, −24.06, 55.44 tokens), further limiting any uniform steering interpretation. The strict deterministic runtime differs from the historical control and prevents attributing the entire observed difference solely to the reward.

The saved compact diagnostic found shorter correct traces within all eight mixed-success groups and small, reinforcing cost-gradient components at the earlier penalty scale. These are post hoc associations. Difficulty can drive both longer traces and failure; they do not identify the causal benefit of changing reasoning length.

5.2 Retrospective diagnosis: reward support, clipping and ceilings

The following analysis was specified after observing the final result. It reconstructs recorded estimator inputs and execution metrics; it adds no model runs, intermediate checkpoint evaluations or confirmatory tests. All inputs are checked against the published evidence manifest. The primary result and its uncertainty calculation remain unchanged.

Why a reward tie prevents an update. For a group of size G, let $\overline{R} = G^{-1}\sum_j R_j$. Starting with the leave-one-out expression and retaining each algebraic step,

$$A_i = R_i - \frac{\sum_j R_j - R_i}{G-1} = \frac{(G-1)R_i - \sum_j R_j + R_i}{G-1} \tag{122}$$

$$= \frac{GR_i - G\overline{R}}{G-1} = \frac{G}{G-1}(R_i - \overline{R}), \tag{123}$$

$$\sum_i A_i = \frac{G}{G-1}\left(\sum_i R_i - G\overline{R}\right) = 0. \tag{124}$$

Define the within-group reward variance with divisor G as $V_R = G^{-1}\sum_i (R_i - \overline{R})^2$. Squaring the centered expression gives

$$\frac{1}{G}\sum_i A_i^2 = \left(\frac{G}{G-1}\right)^2 V_R. \tag{125}$$

Hence all advantages vanish exactly when the rewards are identical. Nonzero reward variance supplies advantage support; it does not ensure a nonzero aggregate gradient, a useful direction, or improved evaluation outcomes. For $R_i = S_i - \lambda C_i$, write $\delta S_i = S_i - \overline{S}$ and $\delta C_i = C_i - \overline{C}$. Directly expanding the square yields

$$R_i - \overline{R} = \delta S_i - \lambda \delta C_i, \tag{126}$$

$$V_R = \frac{1}{G}\sum_i \left((\delta S_i)^2 - 2\lambda \delta S_i \delta C_i + \lambda^2 (\delta C_i)^2\right) \tag{127}$$

$$= V_S - 2\lambda \operatorname{Cov}_G(S, C) + \lambda^2 V_C, \tag{128}$$

where $\operatorname{Cov}_G(S, C) = G^{-1}\sum_i \delta S_i \delta C_i$. In a homogeneous-success group, $\delta S_i = 0$ for every sample, so $V_S = \operatorname{Cov}_G(S, C) = 0$ and $V_R = \lambda^2 V_C$. A positive penalty therefore supplies a nonzero advantage whenever lengths vary, even if all four answers are correct or all are incorrect.

The success-only arm has 30 mixed-success groups, 32 all-correct groups and two all-incorrect groups. It updates on the 30 mixed groups and skips the other 34. The penalty arm has 26 mixed groups, 34 all-correct groups and four all-incorrect groups. It updates on 26 mixed groups and 33 all-correct groups with varying lengths. The remaining all-correct group and four all-incorrect groups have equal costs and skip updates, giving $26+33 = 59$ updates and five skips (Figure 2). The $59-30 = 29$ net difference is not a count of shared trajectories rescued by the penalty: later policies generate different samples. Applying $S - C/1024$ algebraically to the control's recorded rows gives nonzero advantages in 61 groups, but no model was trained on that counterfactual sequence.

An observed identical-trajectory illustration. The first group's task is `v3-t3-train-00367`. Task, seeds, answer choices, scratch token IDs and sampled log likelihoods match exactly across both arms. All four answers are correct and costs are $(1196, 1036, 2049, 2049)$. Thus $\overline{C} = 1582.5$ and, for the penalty,

$$\overline{R} = 1 - 1582.5/1024 = -0.54541015625, \tag{129}$$

$$A_i = \frac{4}{3}(R_i - \overline{R}) = \frac{4}{3}\left(-\frac{C_i - 1582.5}{1024}\right) = \frac{1582.5 - C_i}{768}, \tag{130}$$

$$(A_1, A_2, A_3, A_4) = \left(\frac{773}{1536}, \frac{1093}{1536}, -\frac{311}{512}, -\frac{311}{512}\right) \tag{131}$$

$$\simeq (0.503255, 0.711589, -0.607422, -0.607422). \tag{132}$$

The advantages sum to zero. The loss coefficients $-A_i/4$ favor the likelihood of shorter successful traces and disfavor longer ones. Success-only gives $(0, 0, 0, 0)$ and skips its update. This is an observed estimator difference on identical initial samples; subsequent optimization and performance are not determined by these signs alone.

Raw gradients and clipping. The frozen trainer records the raw norm returned by `clip_grad_norm_`, before scaling gradients at threshold $M = 1$. For raw gradient g and $n = \|g\|_2$, its scaling is

$$\widetilde{g} = \kappa g, \qquad \kappa = \min\left(1, \frac{M}{n + 10^{-6}}\right). \tag{133}$$

All 30 control updates and all 59 penalty updates exceed this threshold. Control raw norms have minimum 58.95, median 111.31 and maximum 216.69; penalty norms have minimum 2.49, median 97.00 and maximum 284.80. Median raw multipliers are about 0.00898 and 0.01031, respectively. These are scaling factors for raw gradients, not the ratio of final Adam parameter steps. Substituting $\widetilde{g}$ into the earlier Adam equations changes both first and second moments. With changing gradients, prior moments, bias correction and ϵ, neither a norm nor an update count establishes the cause of the endpoint. No unclipped-training ablation was executed.

Ceilings and systematically selected repairs. Training ceiling hits are 95/256 for control and 100/256 for penalty; final evaluation hits are 20/64 and 23/64. These are recorded truncation events, not evidence that longer allowed traces would solve more tasks. The matched final outcome cells contain 47 samples correct in both arms, seven correct only under penalty, five correct only under control and five incorrect in both. To avoid choosing attractive prose or extreme token savings, we select the lexicographically first (task ID, sample, seed) in each nonempty cell. Table 2 lists those four illustrations; the local website companion exposes candidate repairs and full recorded traces. The rule is retrospective and the illustrations are not representative effect estimates.

Outcome cell (control, penalty)	Task suffix / sample	Control tokens	Penalty tokens
Incorrect, incorrect	00010 / 1	2049	2049
Incorrect, correct	00004 / 0	2049	2049
Correct, incorrect	00001 / 0	1704	2049
Correct, correct	00000 / 0	1094	2049

Table 2: Fixed-rule retrospective illustrations from task IDs beginning `v3-t3-dev-`. Each selected penalty trace hits the ceiling; that property follows from the stated selection rule and must not be generalized to all samples. The full outcome-cell counts and ceiling rates describe the entire matched evaluation.

6 Historical continuation and qualification

The continuation protocol reuses the verified initial/control evidence and restarts the missing cost-penalty arm from its original initialization. Operational checkpoints every eight groups preserve adapter, optimizer and random state. Actual CUDA resume equivalence must pass before scientific training. The result is labeled a continuation, with one training seed and reused development data. The first continuation failed exact recovery qualification despite identical saved sample rows, with tiny differences in updated adapter and optimizer tensors. A second attempt enables strict deterministic CUDA algorithms before model loading; the exact equality gate remains unchanged. This operational change differs from the retained historical control's execution controls and must be disclosed when interpreting the reused-control comparison. The second attempt passed exact recovery qualification, completed all 64 scientific groups and 64 final evaluation records, and passed an independent Python 3.12 audit using its collected frozen source. The collector and a separate local stream restore verified the entire 1,025,274,422-byte encrypted archive before the final close-out audit. Exact GPU deletion was confirmed by independent provider inventory. These checks establish retained evidence integrity and repeatability for the tested restart; they do not establish a learning benefit.

This historical contrast did not meet the frozen promising-response rule. The separately approved and frozen mechanism study below tests the resulting homogeneous-success support hypoth-

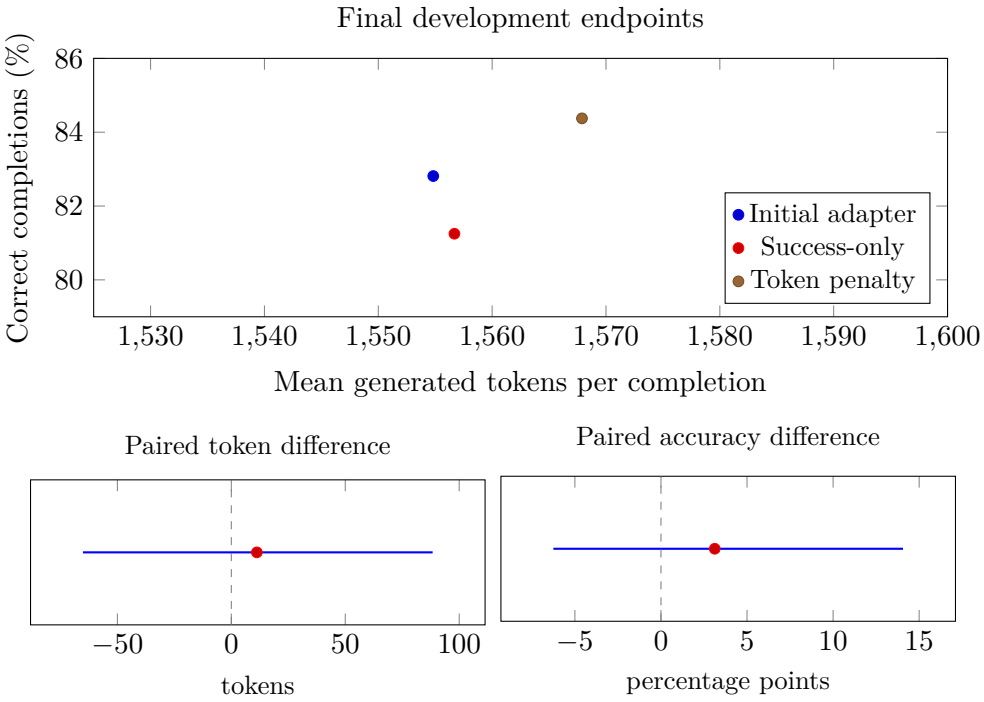


Figure 1: Audited October 10 development comparison, one seed and reused control. Points are final means, not a frontier. Lower panels show penalty-minus-control differences with descriptive 95% paired task intervals, excluding training-seed and design-selection uncertainty. Both intervals include zero.

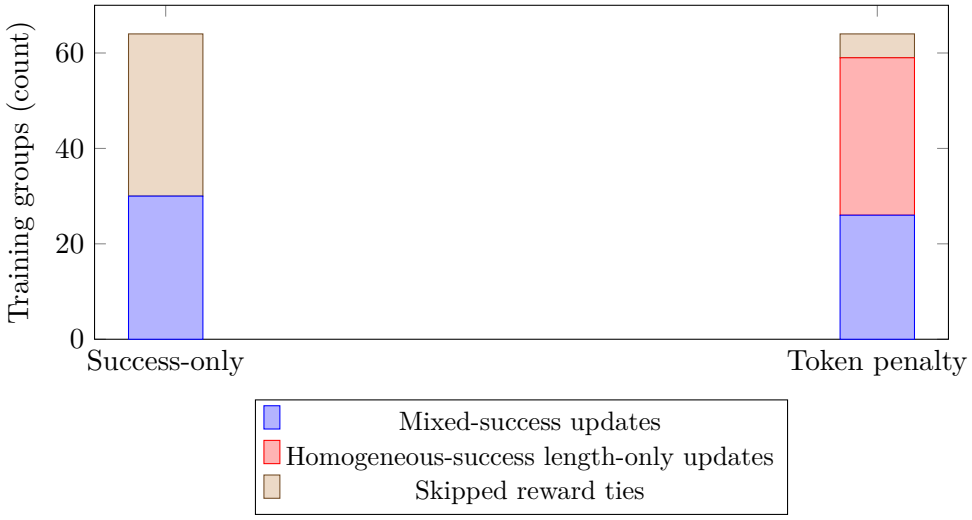


Figure 2: Retrospective reconstruction from all 64 groups per arm. Success-only updates on 30 mixed-success groups and skips 34 ties. The penalty updates on 26 mixed-success groups and 33 homogeneous-success groups with varying lengths, skipping five ties. Both arms receive 256 trajectories. These categories explain update support on recorded samples, not a causal performance effect.

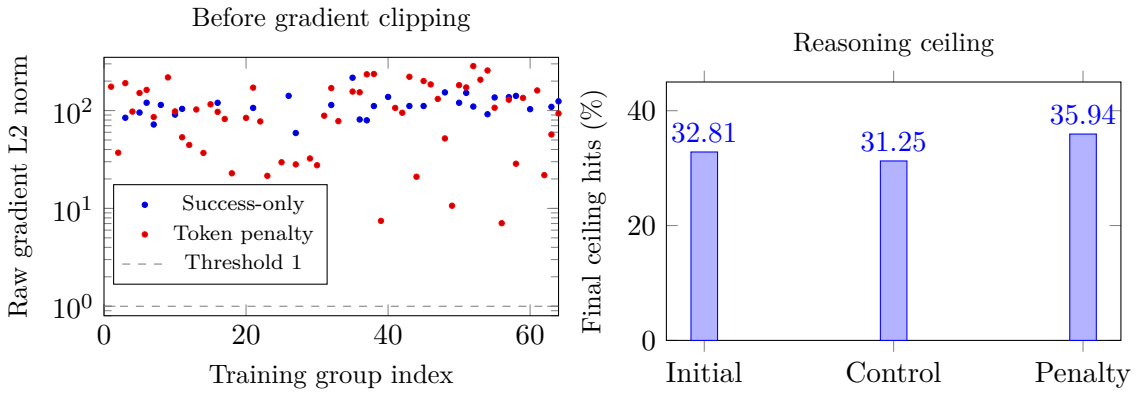


Figure 3: Retrospective clipping and ceiling diagnostics from the audited records. Left: all 89 active updates exceed raw gradient norm 1; the vertical axis is logarithmic and clipping precedes Adam. Right: final ceiling hits are 21/64 initial, 20/64 control and 23/64 penalty. These observations do not identify a cause for the final cost response.

esis with fresh controls and a common deterministic runtime. It does not implement the adaptive controller. No final outcome selected reward scales, prompts, seeds, horizons or checkpoints.

7 Matched three-seed mechanism study

The earlier development result suggested an estimator-support hypothesis: a token penalty can create nonzero advantages in groups whose success indicators are tied. That observation motivated a separately approved experiment, frozen before its final tasks were evaluated. The nine-worker study completed on October 11, 2026 (Pacific time). It did not reuse the historical trained control or pool its outcomes with the one-seed development contrast. The intervention and primary endpoint were chosen from development evidence; the new final set remained untouched during preparation and training.

7.1 Registered task split, allocation and training recipe

The registered generator produced 512 training tasks and 96 final tasks across four expression families: bounded affine, floor composition, modular composition and nested absolute expressions. Each problem supplied four candidate repairs and four visible input/output constraints, followed by scoring on 40 hidden cases. Each visible constraint left two candidates plausible, while the combined constraints identified the target candidate. Candidate semantic identities on the registered finite domain were disjoint from all historical tier-three partitions and from candidates in other new tasks. This is an outcome-blind exclusion rule, not evidence of transfer beyond the restricted language. Generation seeds were 2026101101 and 2026101102 for training and final tasks. The final set contained 24 tasks per family.

Each training seed selected 16 tasks per family from the training pool and shuffled the resulting 64-task schedule. Seeds 103, 211 and 307 were crossed with arms A, B and C. Within each seed block, all arms used the same task schedule and sample-seed rule, but generated their own fresh trajectories under their evolving policies. For one-based group k and zero-based sample i, the training seed was $10^6 s + 1000k + 10i$. A group contained four trajectories from one task, sampled before any update. Each worker saw 256 scientific training trajectories; the cohort therefore contained 576 groups and

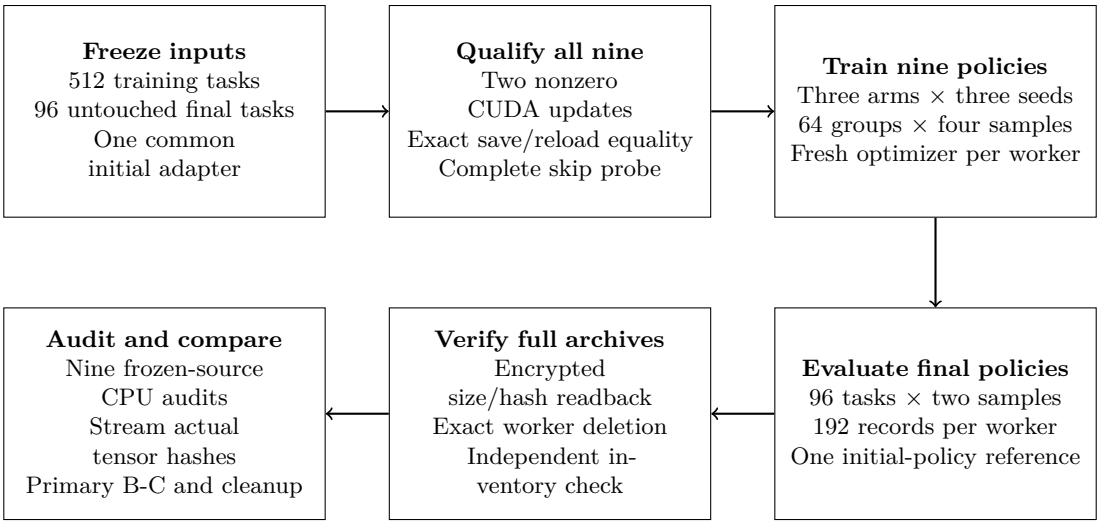


Figure 4: Executed study flow. The final tasks supply evaluation evidence only; they do not select training settings or intermediate checkpoints. Qualification updates are discarded before scientific training. Source: frozen registration, runtime and all nine verified audit/closeout receipts, October 11, 2026.

2,304 training trajectories. The common horizon was 64 groups, not 64 guaranteed optimizer steps.

The sampling policy and differentiated policy use temperature-adjusted likelihoods. The stop event and constrained answer distribution are described in Section 3. Cost counts generated scratch and answer tokens, with a maximum of 2,049; prompt tokens, backward computation, archive transfers and provider spending are excluded from that scientific cost endpoint. A constant learning rate is used. Activation checkpointing reduces memory usage in the backward pass. No intermediate checkpoint is evaluated to choose an endpoint, and no final outcome changes the prompt, reward scale, horizon or seed allocation.

7.2 Three arms and the meaning of a complete mask

For a four-trajectory group, the rewards and raw advantages are

$$R_i^{(A)} = S_i, \qquad R_i^{(B)} = R_i^{(C)} = S_i - C_i/1024, \tag{134}$$

$$A_i^{(a)} = R_i^{(a)} - \frac{1}{3}\sum_{j\neq i} R_j^{(a)} = \frac{4}{3}(R_i^{(a)} - \overline{R}^{(a)}). \tag{135}$$

Let $M_k = \mathbf{1}\{\min_i S_i < \max_i S_i\}$ indicate a mixed-success group. Arm C replaces its raw advantage by $\widetilde{A}_i^{(C)} = M_k A_i^{(C)}$. Its recorded raw rewards and raw advantages are preserved even when masked. When all four answers are correct *or* all four are incorrect, the group performs no backward pass and no optimizer step. Parameters, Adam moments and optimizer counters do not advance through a zero-gradient call. The engineering probe separately checks exact preservation of adapter, nonempty optimizer state and RNG during this skip; drawing the four scientific trajectories still consumes sampling randomness. Arms A and B also skip any exactly tied reward group under the shared implementation.

Setting	Frozen implementation
Backbone	Qwen3-4B-Instruct-2507, revision `cdbee75f17c01a7cc42f958dc650907174af0554`; base weights frozen
Adapter	Identical archived initial adapter in every worker; LoRA rank 8, alpha 16, scale 2, on attention q/k/v/o projections; zero dropout
Precision and hardware	Float32 CUDA, one NVIDIA A40 per worker; no quantization and no model execution on the Mac
Generation	Compact reasoning prompt, temperature 0.7, full categorical sampling; at most 2,048 scratch tokens plus one constrained answer token
Objective	Detached leave-one-out advantages; summed scratch-plus-answer log likelihood; no token-length normalization of the likelihood loss
Optimizer	AdamW with zero weight decay, learning rate 10^{-5}, betas $(0.9, 0.999)$, epsilon 10^{-8}; clip gradient norm at 1
Regularization	No additional KL-reference loss, entropy bonus or adaptive penalty controller in this study
Runtime	Python 3.12.3, PyTorch 2.8.0+cu128, Transformers 4.57.3, PEFT 0.18.0; CUDA 12.8; common manifests and strict deterministic algorithms
Checkpoints	Group 0/8/16/24/32/40/48/56/64, including adapter, optimizer and random state; operational recovery only

Table 3: Executed training recipe. Prompt construction is inherited from the frozen compact-prompt policy wrapper. The common initialization is reloaded after qualification; qualification updates do not enter scientific training.

The intervention is not an unbiased estimator of the original penalty objective. If $h_i = \nabla_\theta \log \pi_\theta(y_i \mid x)$ and $\widehat{g}_B = \frac{1}{4}\sum_i A_i^{(B)} h_i$, the masked estimator has the form $\widehat{g}_C = M_k \widehat{g}_B$ on a hypothetical common group. In general,

$$\mathbb{E}[M_k \widehat{g}_B] = \mathbb{E}[\widehat{g}_B] - \mathbb{E}[(1 - M_k)\widehat{g}_B], \tag{136}$$

and the removed term need not vanish. Once updates differ, B and C sample different trajectories and carry different Adam histories. B-C consequently estimates the final-policy effect of this entire registered optimizer intervention at a common rollout horizon. It does not hold optimizer-step count, trajectory distribution or moment history fixed.

7.3 Qualification, evaluation and paired uncertainty

All nine workers had to pass two actual nonzero CUDA updates with exact uninterrupted versus save/reload agreement before the cohort could release scientific training. A synthetic homogeneous-success probe, using nonempty Adam state, tested both all-correct and all-incorrect optimizer skips without producing scientific task records. Package manifests had to match. Every worker then reloaded the same initial adapter and a fresh optimizer. The replay and loss tolerances in Section 3 stayed unchanged: 0.10 maximum and 0.02 mean scratch discrepancy, 0.02 answer discrepancy in nats, finite gradients, and the recorded loss reconstruction bound. These checks qualify the tested execution path; the later CPU audit binds the actual checkpoint hashes and reconstructs records without rerunning a model.

Only the final group-64 policy was evaluated. Each worker sampled the same 96 final tasks twice, giving 192 records per worker and 1,728 final trained-policy records overall. Evaluation used $48103107 + 100j + 10m$ for zero-based task j and sample m, shared across arms and training seeds. A-seed103 additionally reloaded the original adapter and produced one 192-record initial-policy

reference with the same tasks and evaluation seeds. That reference is a single measurement, never nine independent baseline runs or a source of training feedback.

The primary contrast was B-C mean generated tokens, with solve-rate difference reported alongside it. B-A and C-A were secondary. Write $Y_{a,s,f,j,m}$ for cost or success at arm a, training seed s, family f, within-family task j and sample m. The endpoint averages three seeds, four equally sized families, 24 tasks per family and two samples:

$$\widehat{\Delta}_{a-b,Y} = \frac{1}{3 \cdot 4 \cdot 24 \cdot 2} \sum_{s=1}^{3} \sum_{f=1}^{4} \sum_{j=1}^{24} \sum_{m=1}^{2} (Y_{a,s,f,j,m} - Y_{b,s,f,j,m}). \tag{137}$$

The frozen bootstrap uses 10,000 replicates and seed 8601. Each replicate resamples three paired training-seed blocks with replacement and, separately, 24 task clusters with replacement within each family. The same seed and task draws apply to every arm, retaining both samples. Percentile endpoints are the 2.5% and 97.5% quantiles. These are descriptive crossed seed/task intervals. With only three training seeds, they do not supply precise population seed uncertainty. Repeated evaluations of the same 96 tasks do not create 576 distinct tasks per arm. Secondary intervals are not adjusted for multiple comparisons and do not establish an accuracy-noninferiority claim.

The registered promising-cost-response rule required all three conditions: at least a 10% B-C relative token reduction, an entirely negative token interval, and a negative B-C token difference in every training seed. This was a practical decision rule, not a guarantee of calibrated statistical significance or a trigger for an automatic extension.

7.4 Final outcomes and the registered decision

Arm	Successes	Solve rate	Mean tokens	Updates	Ceiling hits
A	439/576	76.22%	1625.20	96	242/576
B	442/576	76.74%	1609.05	174	245/576
C	419/576	72.74%	1638.62	95	255/576

Table 4: Trained-policy endpoints over three seeds, each evaluated on the same 96 tasks with two samples. Updates sum three 64-group runs; ceiling hits mean 2,049 generated tokens. Denominators are repeated task/sample slots, not independent unique tasks.

Arm	Seed	Successes	Mean tokens	Updates	Masked groups	Ceiling hits
A	103	145/192	1633.23	29	0	85/192
A	211	145/192	1614.31	28	0	81/192
A	307	149/192	1628.07	39	0	76/192
B	103	146/192	1594.17	61	0	83/192
B	211	142/192	1616.01	57	0	87/192
B	307	154/192	1616.98	56	0	75/192
C	103	137/192	1644.45	31	33	86/192
C	211	138/192	1625.19	27	37	85/192
C	307	144/192	1646.21	37	27	84/192

Table 5: All nine registered workers, with no dropped seed or selected checkpoint. Every worker completed 64 groups and 192 final evaluation records.

Contrast	Δ tokens	Descriptive interval	Δ accuracy (pp)	Descriptive interval (pp)
B-A	-16.15	[-58.62, +25.32]	+0.52	[-4.34, +4.86]
B-C	-29.56	[-76.66, +15.48]	+3.99	[-0.52, +8.51]
C-A	+13.41	[-12.00, +40.84]	-3.47	[-6.42, -0.69]

Table 6: Registered primary B-C and secondary contrasts. Accuracy is in percentage points; intervals are descriptive 95% crossed seed/task bootstrap intervals. Negative token differences favor the first arm.

The primary B-C mean difference was -29.564236 tokens, or -1.804218% relative to C. Its descriptive interval, [-76.662413, +15.480469], includes zero. B-C accuracy was +3.993056 percentage points, with interval [-0.520833, +8.506944]. The per-seed token differences were -50.281250, -9.187500 and -29.223958 for seeds 103, 211 and 307. Directional agreement satisfied the third decision condition; the 10% magnitude and entirely negative interval conditions failed. The registered promising-cost-response flag was therefore false. This is a negative practical decision and an uncertain small effect, not evidence that every possible benefit is absent.

B-A averaged -16.151042 tokens and +0.520833 accuracy percentage points, with both intervals spanning zero. C-A averaged +13.413194 tokens and -3.472222 accuracy percentage points. Its secondary accuracy interval [-6.423611, -0.694444] lies below zero, but this is an unadjusted descriptive contrast in a small study; it is not a broad claim that masking universally harms accuracy. The initial-policy reference scored 144/192 (75.00%) at 1,624.265625 mean tokens, with 85 ceiling hits. The trained A, B and C means were 76.22%, 76.74% and 72.74%. No independent multi-seed initial-policy uncertainty is available, so these baseline comparisons remain descriptive.

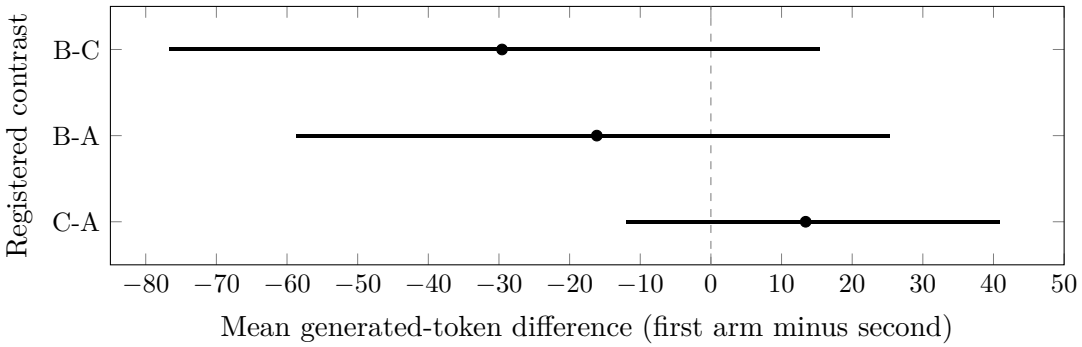


Figure 5: Mean final-token contrasts and descriptive 95% crossed seed/task intervals from the frozen 10,000-replicate analysis. Points are means; horizontal lines are intervals; the dashed line is zero. Every token interval crosses zero. Source: all nine independently audited group-64 evaluations, October 11, 2026.

7.5 Measured update support, ceilings and interpretation

Across 192 training groups per arm, A updated on 96 mixed-success groups and skipped 96 homogeneous groups (88 all-correct, eight all-incorrect). B updated on 174 groups: 91 mixed-success and 83 homogeneous-success groups; 18 homogeneous groups had exactly tied token costs and were skipped. C updated on its 95 mixed-success groups and masked 97 homogeneous groups (87 all-correct and ten all-incorrect). Thus B took more updates than either A or C, without meet-

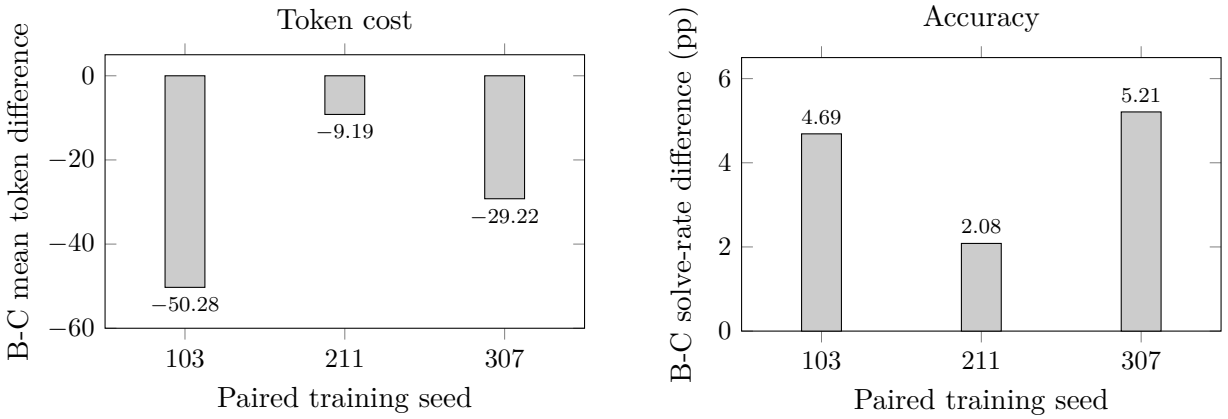


Figure 6: Primary B-C endpoints separately for each training seed. Each bar compares two final policies on the same 96 tasks and two evaluation samples per task. Negative tokens and positive accuracy favor B. Bars are observed point differences, with no per-seed uncertainty bars; the registered pooled descriptive intervals remain in Figure 5. Directional agreement across three seeds does not imply a calibrated population effect or satisfaction of the 10% practical rule. Source: all nine frozen independent audits, October 11, 2026.

ing the final token-response criterion. B's 83 homogeneous updates are reconstructed on B's own trajectories; they are not 83 common groups rescued relative to C, because the policies diverge.

Every one of the 365 active updates had a raw gradient norm above one and was clipped. This count is a retrospective diagnostic from saved metrics, not an experimental manipulation of clipping. Final ceiling hits were 242/576 for A, 245/576 for B and 255/576 for C (42.01%, 42.53% and 44.27%). Frequent ceiling hits constrain how token-length changes can appear, but neither the ceiling counts nor universal clipping identify the cause of the endpoint. Changing the ceiling, reward scale, optimizer or training horizon after inspecting these outcomes would define a new experiment. There is no checkpoint performance curve: saved intermediate checkpoints serve recovery, and only final policies were evaluated.

7.6 Recovery history, resource accounting and audit boundary

The first nine-worker provisioning attempt failed before model execution: eight create requests failed and one created worker failed SSH readiness. Its verified failure metadata, deletion receipt and accounting were preserved; no scientific run from that attempt enters the comparison. A separately approved second cohort supplied all nine completed workers. Each group, task and checkpoint had to receive an encrypted off-Pod size/hash acknowledgement bound to the exact worker, freeze, sequence and manifest before dependent work continued. Three shared transfer slots prioritized recovery over terminal collection. Full terminal archives were verified before early deletion; independent immutable workload and deletion guards stayed in force. Ordinary workload/deletion limits were 19.5/20.25 hours after creation, and the reference worker's were 27.3/28.05 hours.

Three workers required additional terminal archive attempts after transfer or verification failures. Scoped approvals increased only their attempted-transfer reservations within the original 20-billion-byte evidence ceiling; there was no replacement worker, retraining, horizon extension, gate relaxation or extra storage purchase. Failed attempt evidence and the prior ledger were retained. All nine terminal archives ultimately passed full byte-count and SHA-256 readback. The finalizer selectively restored bounded text and streamed the actual hashes of 25 tensor members per

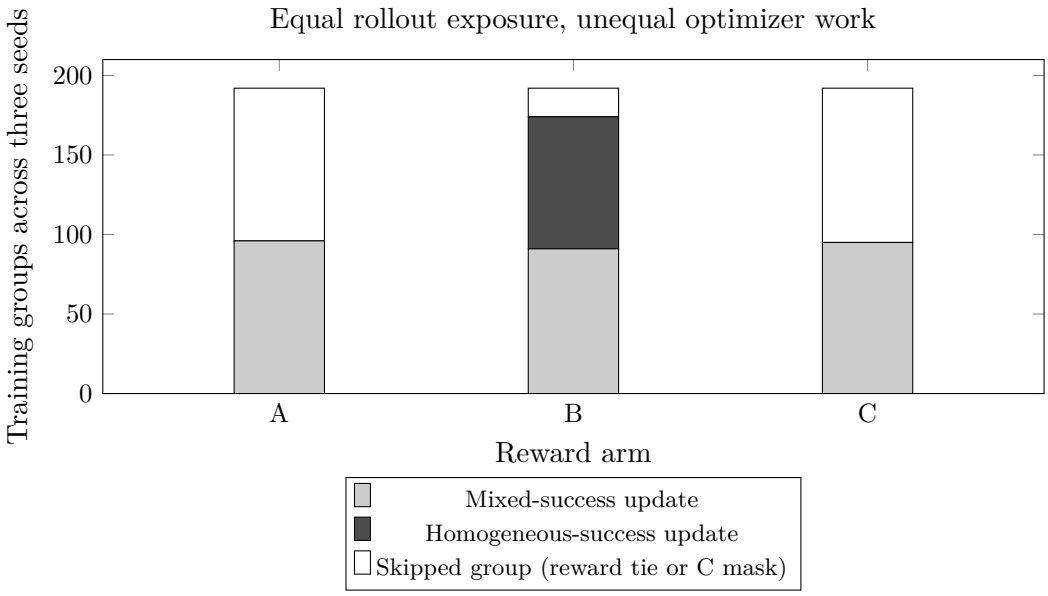


Figure 7: Update support over 192 groups per arm (three runs of 64 groups). A takes 96 updates, B 174 and C 95; the remaining groups still generate four trajectories each. A and B skips are exact reward ties, whereas C's 97 skips are homogeneous-success masks. The 83 B homogeneous updates occur on B's own evolving trajectories, not shared groups that can be counted as rescued relative to C. Source: 576 independently audited training-group metrics, October 11, 2026.

worker without storing a model on the Mac. Independent Python 3.12 audits from each collected frozen source reconstructed rewards, advantages, masks, update counts, replay/loss checks, paired identities, checkpoint bindings and final outcomes. All nine complete audits were required for the comparison; a recovery snapshot alone would have remained explicitly incomplete.

The conservative cumulative GPU/disk plus storage-reserve estimate was $99.79561 under the approved $131 cap, including $1.01185 carried from the failed first attempt. The nominal aggregate billed worker lifetime was 163.79190 single-GPU hours; summed GPU-hours are distinct from elapsed wall time. Posted provider billing for the second cohort was $97.23842 at closeout and could lag; it excludes the separately carried first-attempt estimate. Scientific token cost is a different quantity. Independent inventory confirmed all study workers absent, all 30 study services were disabled and unloaded, detached deadline processes were absent, and only the temporary study credential copy was removed. Historical evidence, original credentials and encrypted objects were preserved. The old API-caller/support investigation remains unresolved and is not attributed to this completed cohort.

The study freeze was `67301c1aadbecb08ce230bbc0db6d736da73cfa9163e2230a9f9d267272ca324`. The public scientific-input manifest binds the raw records and frozen source to their original hashes. The original registration, generator provenance and all failed evidence are retained. The reviewed public subset contains the completed scientific records and CPU reconstruction tools; full model tensors and operational records remain private.

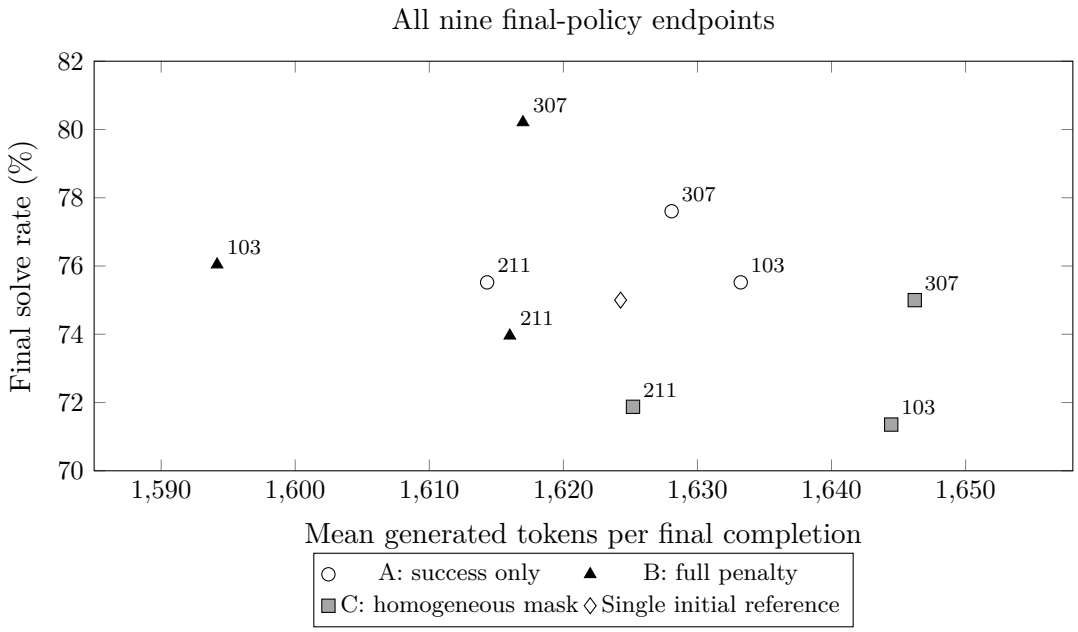


Figure 8: Final generated-token cost and solve rate for every arm/seed, with seed labels beside the points. Each trained-policy point uses 192 matched evaluation records on 96 final tasks. The diamond is the one 192-record initial-policy reference, not three independent baselines. Lower token cost and higher solve rate are preferred. Points are not connected: these are final observed policies, not an estimated Pareto frontier, training trajectories or confidence regions. Source: all nine frozen independent audits and the registered initial reference, October 11, 2026.

8 Limitations

The synthetic candidate-selection task is a limited proxy for coding. The historical one-seed development split was inspected repeatedly and has a runtime difference between its retained control and continuation. The new mechanism study addresses those specific weaknesses with fresh controls, matched deterministic runtimes, three seeds and a previously untouched final set; it remains motivated by the earlier development result and confined to four familiar expression families. Semantic exclusion on a finite domain does not establish repository-level repair or out-of-language generalization. Three seeds give limited information about optimizer variability, and the descriptive crossed bootstrap is not a precision guarantee or a multiple-testing correction.

The initial reference is measured once. The arms share final tasks and sample seeds; their 576 records per arm are repeated observations on 96 tasks. Only group-64 policies are evaluated. A common rollout horizon leaves unequal optimizer-step counts and Adam histories. In particular, the C mask tests a complete optimizer intervention and cannot identify a pure cost-gradient contribution while holding everything else fixed. Universal clipping and frequent token ceilings are measured diagnostics, not experimentally isolated explanations. Tokens measure generated length, not total computation or provider spending. The registered negative decision does not prove zero effect or justify another automatic run. Neither completed study establishes an adaptive-controller advantage or a Pareto frontier.

Two cloud interruptions required a missing-arm continuation; the fixed-reward comparison is now complete. An October 9 provider reply attributes both deletions to account API-key requests through the API/SDK, ruling out provider infrastructure, machine issues, balance exhaustion and template TTL. The responsible key and calling process remain unidentified. The user authorized disabling all previously active API keys and resuming through fresh isolated study access. The continuation run log records its approved $10 cap, credential mitigation and completed continuation. The support investigation remains open. Recovery and numerical validation support evidence integrity but do not establish a performance improvement.

9 Reproducibility and release

The versioned public release, `arpjw/reward-function`, `research-v1.1.0`, contains the paper, derivations, historical development evidence and the completed nine-worker mechanism study. The website visualizer reports both studies separately and links to this same version. Earlier immutable releases remain available for the historical record.

For the mechanism study, the executed method, result report, scientific evidence package and CPU reconstruction program provide the registration, original generator, frozen scientific source/configuration, complete recorded training/evaluation trajectories, per-group metrics and independent audit receipts. Compressed record members retain their original decompressed bytes; the scientific-input manifest records their SHA-256 and size. No task is regenerated for publication. Public analysis independently reconstructs scoring, reward/advantage and mask arithmetic, update support, numerical gates, paired identities, final endpoints and the registered bootstrap without a model or account credentials.

The public scope statement distinguishes this CPU reconstruction from the execution-time CUDA save/reload test and the private full archive/tensor audit. Public checkpoint metadata and recorded tensor hashes bind the retained audit receipts; readers cannot independently rehash absent model/optimizer/RNG tensors from the public subset. Credentials, provider account/support records, cloud-control receipts and full encrypted operational archives are excluded. The public file

manifest binds the released source, records, paper and derived figures to their reviewed bytes.

10 Conclusion

The historical development study showed that token penalties can activate otherwise tied groups without demonstrating token savings. The subsequent matched three-seed mechanism study preserved that update-support distinction: the full penalty arm took 174 updates versus 96 for success-only and 95 for the homogeneous-success mask. On previously untouched final tasks, full penalty used 29.56 fewer tokens than the masked arm, with the same direction in all three seeds, but its descriptive interval crossed zero and its 1.80% reduction failed the registered 10% promising-response rule. The penalty-versus-success-only token interval also crossed zero. The completed result therefore supports a bounded report of the tested mechanism and a negative practical decision; it does not demonstrate a useful token-saving method or prove that a small effect is absent.

Fresh controls, common deterministic execution, complete encrypted archives and nine independent frozen-source audits strengthen the evidence boundary relative to the earlier continuation. Three seeds, synthetic candidate selection, clipping, ceilings and an intervention that changes Adam history still limit causal and generalization claims. An adaptive controller remains untested in this completed mechanism study. The mathematical derivations, measured endpoints and retained failures explain what was tested and what remains uncertain.

A Full gradient through computation history

Equation (47) requires every scratch conditional's parameter gradient. A cached autoregressive computation can be written as

$$h_0 = P_\theta(x), \qquad h_{k+1} = F_k(h_k, \theta), \qquad \mathcal{L}_{\text{scratch}} = a \sum_{k=0}^{B-1} l_k(h_k, \theta), \tag{138}$$

where l_k is a block's log likelihood for fixed recorded tokens, and $a = -A_i/G$ is its detached scalar coefficient. Blocks summarize the computation graph, not independently sampled actions. Here h_k is an internal computational state, distinct from the trajectory score h_θ in Section 3. Define the remaining loss as a function of the current state:

$$\Phi_B(h_B, \theta) = 0, \qquad \Phi_k(h_k, \theta) = a\, l_k(h_k, \theta) + \Phi_{k+1}(F_k(h_k, \theta), \theta). \tag{139}$$

At a state boundary, $d_k = \nabla_{h_k} \Phi_k$ collects the future-loss dependence on that state. The chain rule applied to the second term multiplies its downstream gradient d_{k+1} by the transpose of the state-transition Jacobian $\partial_{h_k} F_k$. The direct parameter branch of the same transition contributes $(\partial_\theta F_k)^\top d_{k+1}$. Adding each local likelihood branch gives

$$d_B = 0, \tag{140}$$

$$d_k = a\, \partial_{h_k} l_k + (\partial_{h_k} F_k)^\top d_{k+1}, \tag{141}$$

$$g_k = a\, \partial_\theta l_k + (\partial_\theta F_k)^\top d_{k+1}, \tag{142}$$

$$\nabla_\theta \mathcal{L}_i = \sum_k g_k + (\partial_\theta P_\theta(x))^\top d_0 + a\, \nabla_\theta \log p^{\mathcal{A}}_{\theta,T}(a_i^{\text{ans}} \mid r(x, Z_i)). \tag{143}$$

The readout is a separate re-encoded computation graph. Its parameter gradient is added separately; it does not supply a cotangent to the scratch cache through a discrete decode operation. Setting

intermediate d_k to zero would discard history terms and would be truncated backpropagation, a different estimator. The terminal condition $d_B = 0$ follows because no scratch loss remains after the last block. Summing g_k counts the direct parameter branches at every block, while $(\partial_\theta P_\theta(x))^\top d_0$ accounts for parameter dependence in the initial prompt computation. These are all branches of the scratch graph; adding the separate answer gradient completes the per-trajectory loss derivative.

The historical MLX implementation explicitly propagated cache-boundary cotangents. The current CUDA implementation uses a full teacher-forced scratch graph with activation checkpointing and a separate readout backward pass. Recomputing activations can reduce memory while retaining the chain rule, provided it reproduces the same computation. Numerical replay tolerances quantify observed approximation, not an analytic equality certificate for every gradient. Results from distinct numerical backends are not pooled as if their policies were identical.

B Checkpoint state, exact recovery and durable evidence

A recoverable state contains more than adapter weights. Write

$$\mathcal{S}_k = (\theta_k, m_k, v_k, t_k, k, \mathcal{R}_k, \mathcal{P}_k, \mathcal{I}), \tag{144}$$

where t_k is the optimizer's update count, k is the completed group position, $\mathcal{R}_k$ contains Python, NumPy and Torch CPU/CUDA random states, $\mathcal{P}_k$ authenticates record prefixes and named parameter ordering, and $\mathcal{I}$ binds source, inputs, task schedule, sampling seeds and runtime. The optimizer's per-parameter step state is preserved. A notional update transition is

$$(\mathcal{S}_{k+1}, \mathcal{O}_{k+1}) = F_{\omega,k}(\mathcal{S}_k), \tag{145}$$

where ω includes the generating runtime and numerical-kernel controls, and $\mathcal{O}$ denotes sampled records and update metrics. A deterministic transition is an assumption about execution controls, not a consequence of recording seeds alone.

The mandatory CUDA qualification saves state after the first prescribed group, computes group two uninterrupted, reloads group-one state into a fresh model and optimizer, then repeats group two. It requires

$$\mathcal{O}_2^{\text{uninterrupted}} = \mathcal{O}_2^{\text{reloaded}}, \tag{146}$$

$$\theta_2^{\text{uninterrupted}} = \theta_2^{\text{reloaded}}, \tag{147}$$

$$(m_2, v_2, t_2, \mathcal{R}_2)^{\text{uninterrupted}} = (m_2, v_2, t_2, \mathcal{R}_2)^{\text{reloaded}}. \tag{148}$$

Adapter serialization hashes must match; optimizer/RNG structures and tensors are compared exactly. Both prescribed updates must be nonzero. The qualification's repeated trajectories are not added to scientific training.

The first continuation failed this exact test: sample rows matched, but small adapter/optimizer differences remained. The second uses strict deterministic CUDA controls and passed the unchanged test. This establishes repeatability for the tested restart on its recorded runtime. It does not prove bitwise equivalence for every future restart, another architecture or another backend. The retained success-only control did not use the new bootstrap controls; that operational difference limits the reused-control contrast.

Scientific checkpoints at $k \in \{0, 8, 16, \ldots, 64\}$ bind the adapter, optimizer, random state and exact prefix. Every completed group and paired evaluation task is protected by an immutable recovery manifest before the next unit of work. For each included file f, retain

$$\mathcal{M}[f] = (\text{SHA256}(f), \text{bytes}(f)). \tag{149}$$

The encrypted off-Pod object is read back in full and checked against both fields. An acknowledgement must bind the same Pod ID, source freeze, sequence and manifest hash. Missing or mismatched acknowledgement blocks further sampling. A saved weight file without optimizer state or a verified record prefix is not a qualified resume point. An archived partial result does not constitute a completed comparison.

C What is established and what remains empirical

The mathematical statements in this draft have distinct scopes:

1. Equations (5)–(13) give an exact algebraic scalarization for interior weights with positive, fixed baseline references.

2. Equations (18)–(28) solve a strictly convex one-dimensional dual step while treating gain estimates as fixed. They do not prove convergence of the coupled learning process.

3. Equation (57) is conditionally unbiased under independent current-policy samples, fixed penalty and exact scores. Clipping, Adam, finite precision and a changing penalty prevent interpreting every parameter update as that ideal expectation.

4. Equations (97)–(99) and the exact restart check are implemented numerical gates. Passing them supports evidence integrity, not a success-cost improvement.

5. Equation (111) describes the frozen one-seed development analysis. It does not quantify training-seed variation or remove repeated development selection.

6. The fixed-reward development comparison and the matched three-seed mechanism study are complete and independently audited. Neither meets its registered practical token-response rule. The mechanism study uses a separate crossed seed/task analysis and tests a masked optimizer intervention rather than the ideal unbiased penalty gradient. A reliable reward response, an adaptive-rule advantage and a Pareto-frontier claim require increasingly broader empirical evidence; none of those stronger claims is established here.

The full derivation and experiment-specific qualifications are also preserved in MATH.md, the historical RL method, the fixed-contrast protocol and the historical continuation run log. Preparation status in a historical protocol is not a current execution receipt.

References

[1] Arash Ahmadian et al. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740, 2024.

[2] Shih-Yang Liu et al. DLER: Doing Length pEnalty Right: Incentivizing More Intelligence per Token via Reinforcement Learning. arXiv:2510.15110, 2025.

[3] Anish Lakkapragada. Extending SWE-2's reward function to steer the Pareto frontier. 2026.

[4] Cognition. Introducing SWE-2: Pushing the Pareto Frontier. 2026.

[5] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[6] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.

[7] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. ICLR, 2015.

[8] Qwen. Qwen3-4B-Instruct-2507 model card.

[9] Project evidence: CUDA pilot, compact diagnostic, reward geometry, and recovered retry.